

06. Features Descriptors

Jae-Young Choi

Sungkyunkwan University

Outline



- Feature Description
- Feature Descriptors
 - SIFT, SURF, HOG
 - BRIEF, ORB

Feature Description

Describe a local region



Step 1: extract features → Step 2: match features → Step 3: align images

Characteristics of good features

- Repeatability

- The same feature can be found in several images despite geometric and photometric transformations

- Saliency

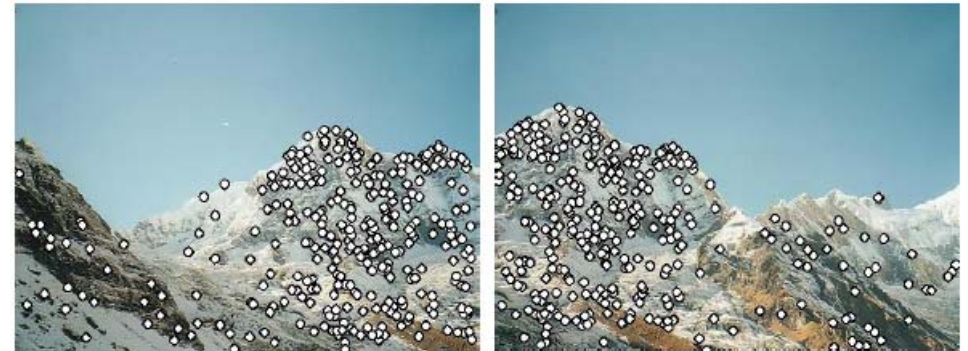
- Each feature is distinctive

- Compactness and efficiency

- Many fewer features than image pixels

- Locality

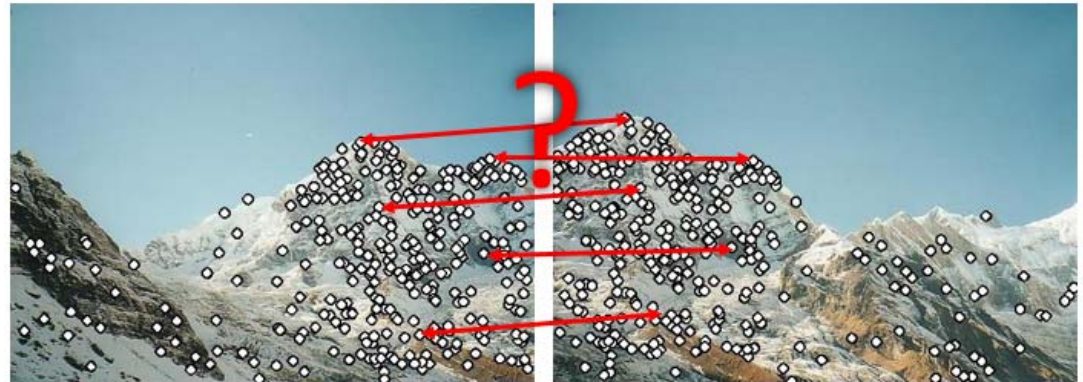
- A feature occupies a relatively small area of the image; robust to clutter and occlusion



Describe a local region

- We know how to detect points
- Next question

- How to match them?



Describe a local region

- Feature Matching with templates
 - Given a template - find the region in the image with
 - the highest matching score
- Given rotated, scaled, perspectively distorted version of the image
- Can we find the same patch (we want invariance!)
 - Scaling - NO
 - Rotation - NO
 - Illumination - depends
 - Perspective Projection - NO

Describe a local region

- How to match them?
 - **Answer:** Come up with *a descriptor* for each point,
find similar descriptors between the two images
- Point descriptor should be
 - Invariant
 - Distinctive

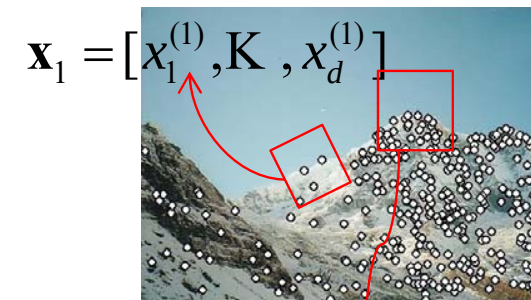
How detect/extract features?

- Local features: main components

- Detection:** Identify the interest points

- Description:** Extract vector feature descriptor surrounding each interest point.

- Matching:** Determine correspondence between descriptors in two views



$$\mathbf{x}_1 = [x_1^{(1)}, K, x_d^{(1)}]$$

$$\mathbf{x}_2 = [x_1^{(2)}, K, x_d^{(2)}]$$



Descriptors ?

- Image Patch

- Just use the pixel values of the patch

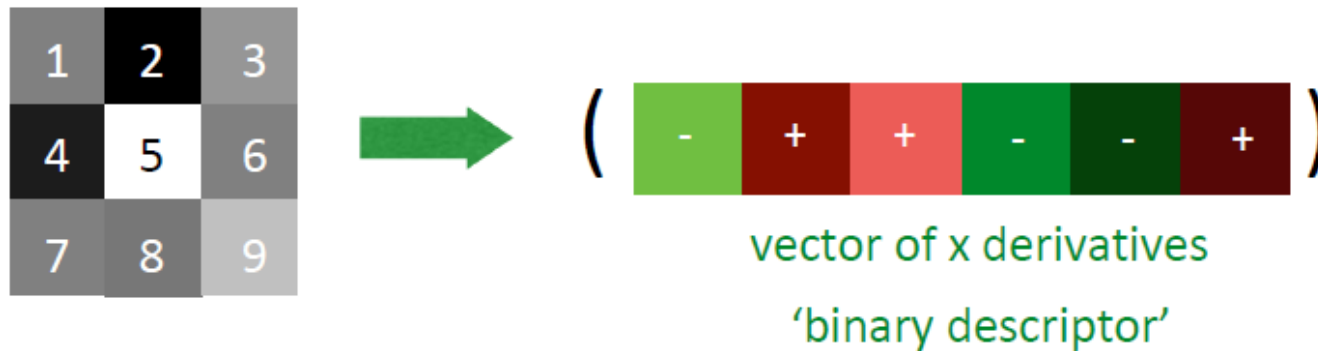


- Perfectly fine if geometry and appearance is unchanged
- *What are the problems?* ***It is sensitive to absolute intensity values***

Descriptors ?

■ Image Gradients

- Use pixel differences



- Invariant to absolute intensity values
- *What are the problems?* ***It is sensitive to deformations***

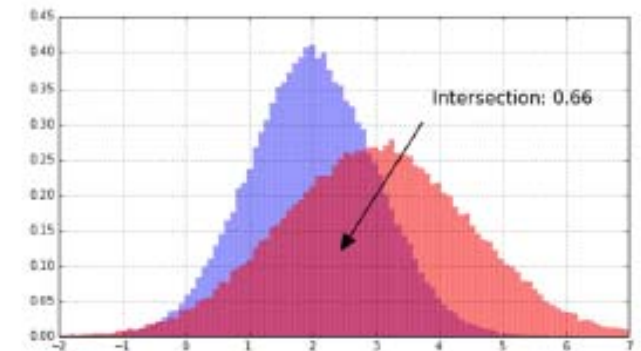
Descriptors ?

■ Color Histogram

- Count the colors in the image using a histogram
- Similarity is measured based on histogram intersection:



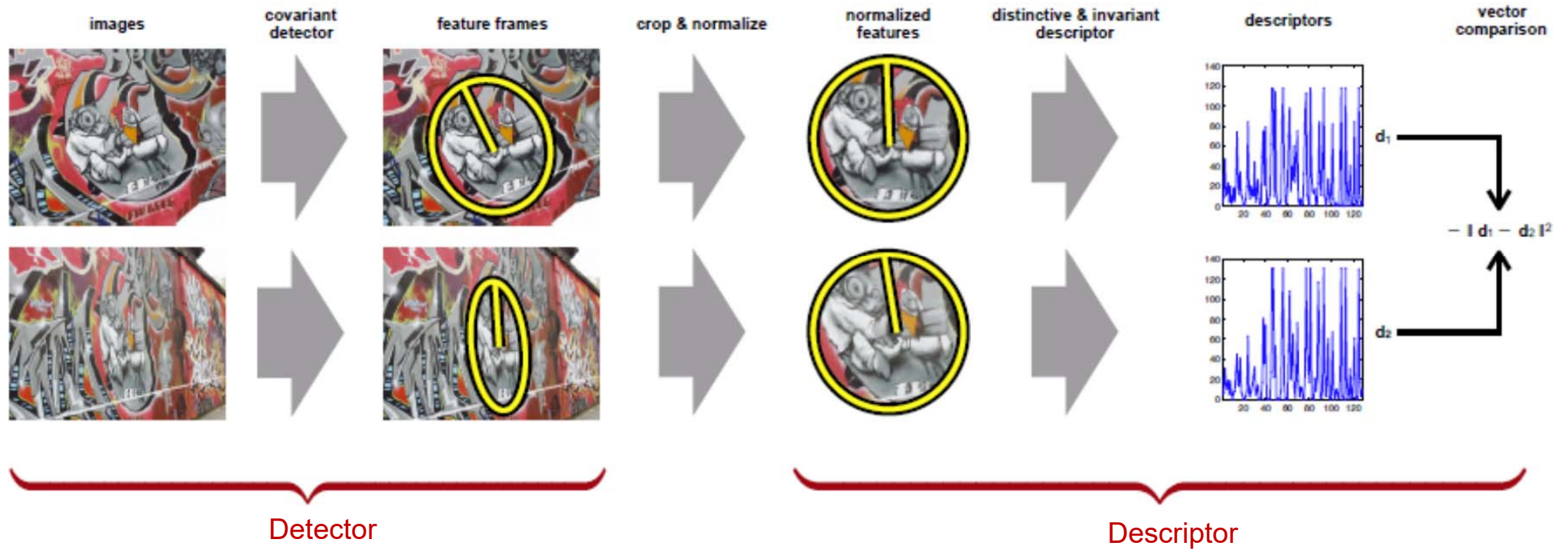
$$s_{\text{color}}(r_i, r_j) = \sum_{k=1}^n \min(c_i^k, c_j^k)$$



- Invariant to changes in scale and rotation
- *What are the problems?*

Different images may have same histogram

Descriptors ?



SIFT

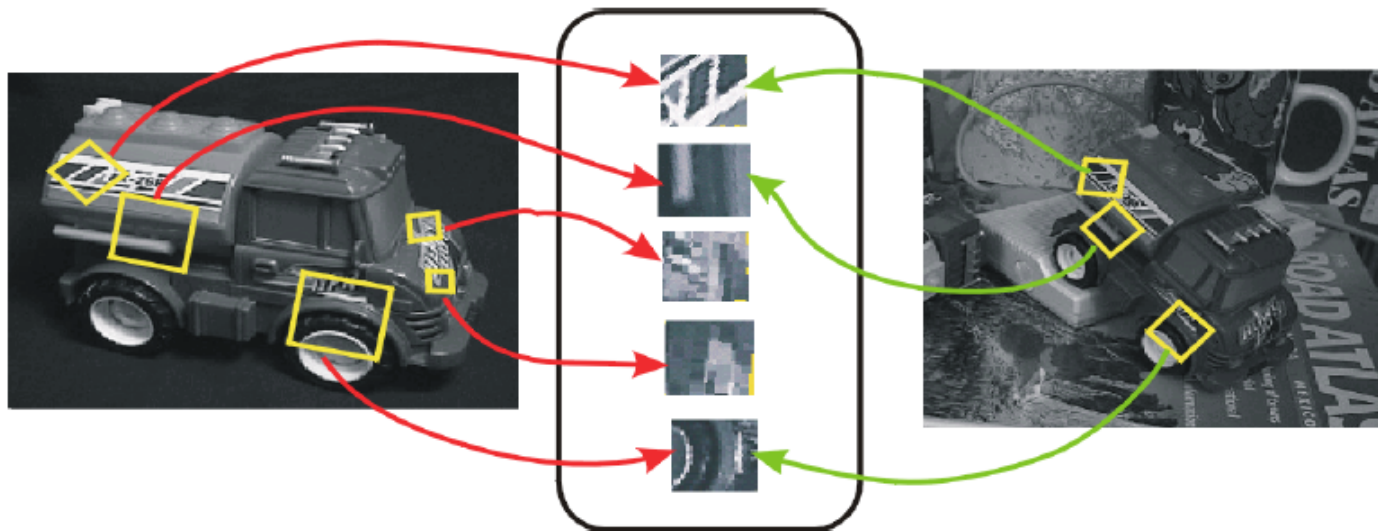
SIFT = Scale Invariant Feature Transform

Distinctive image features from scale-invariant keypoints. David G. Lowe, International Journal of Computer Vision, 60, 2 (2004), pp. 91-110.

SIFT Features

- Invariant Local Features

- Image content is transformed into local feature coordinates that are invariant to translation, rotation, scale, and other imaging parameters



SIFT Features

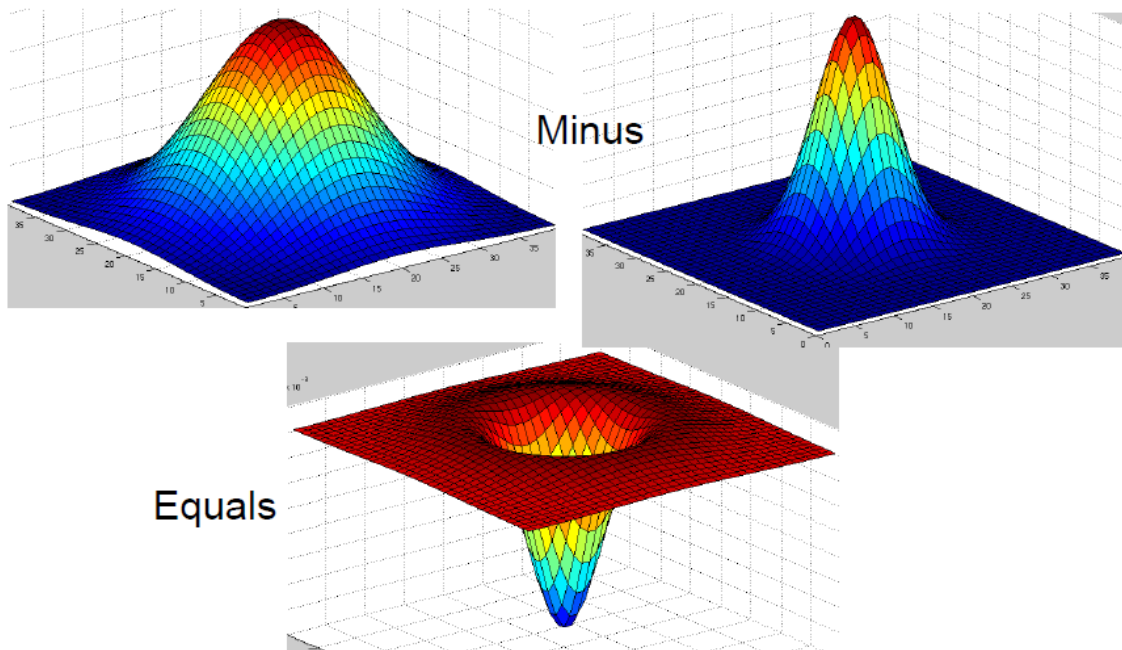
1. **Enforce invariance to scale:** Compute Gaussian difference max, for many different scales; non-maximum suppression, find local maxima: keypoint candidates
2. **Localizable corner:** For each maximum fit quadratic function. Compute center with sub-pixel accuracy by setting first derivative to zero.
3. **Eliminate edges:** Compute ratio of eigenvalues, drop keypoints for which this ratio is larger than a threshold.
4. **Enforce invariance to orientation:** Compute orientation, to achieve rotation invariance, by finding the strongest second derivative direction in the smoothed image (possibly multiple orientations). Rotate patch so that orientation points up.
5. **Compute feature signature:** Compute a "gradient histogram" of the local image region in a 4x4 pixel region. Do this for 4x4 regions of that size. Orient so that largest gradient points up (possibly multiple solutions). Result: feature vector with 128 values (15 fields, 8 gradients).
6. **Enforce invariance to illumination change and camera saturation:** Normalize to unit length to increase invariance to illumination. Then threshold all gradients, to become invariant to camera saturation.

SIFT Features

1) Keypoint detection

- Find keypoints (extrema) using DoG (approximated LoG)

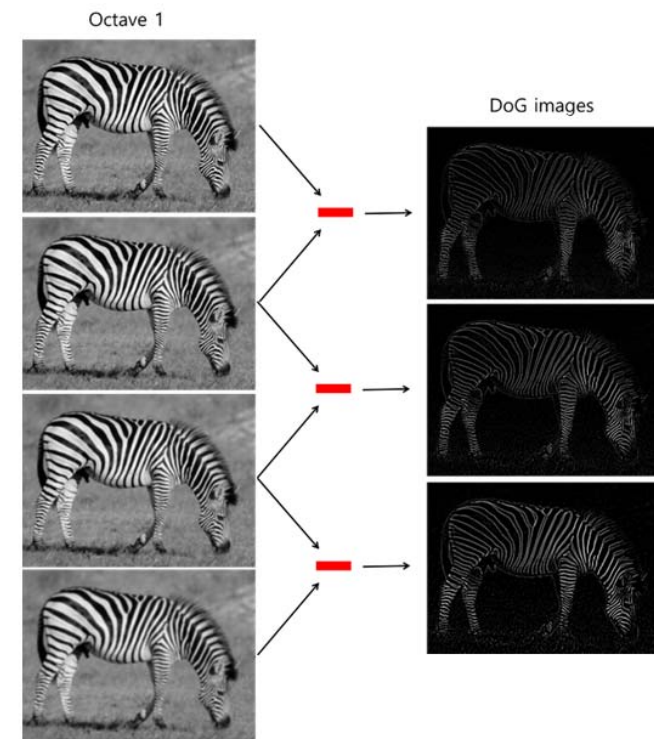
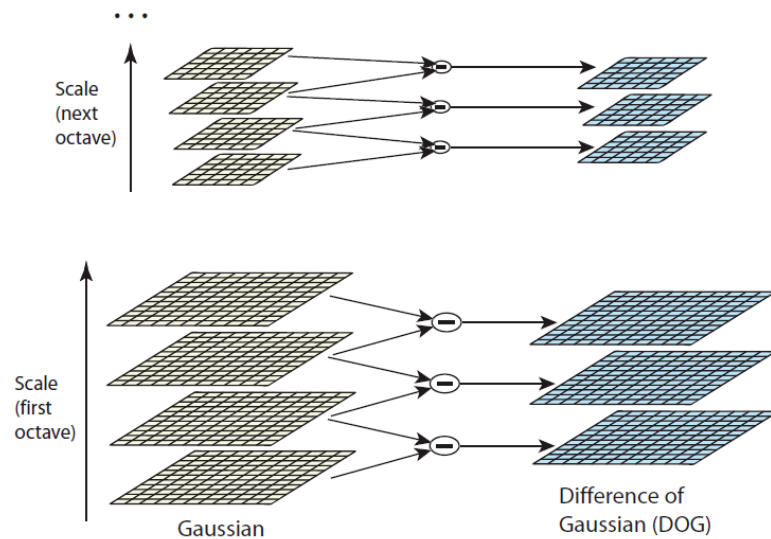
Difference of Gaussians



SIFT Features

■ Keypoint detection

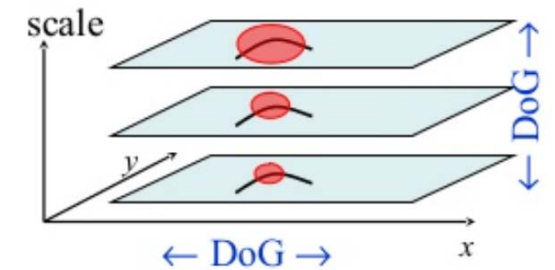
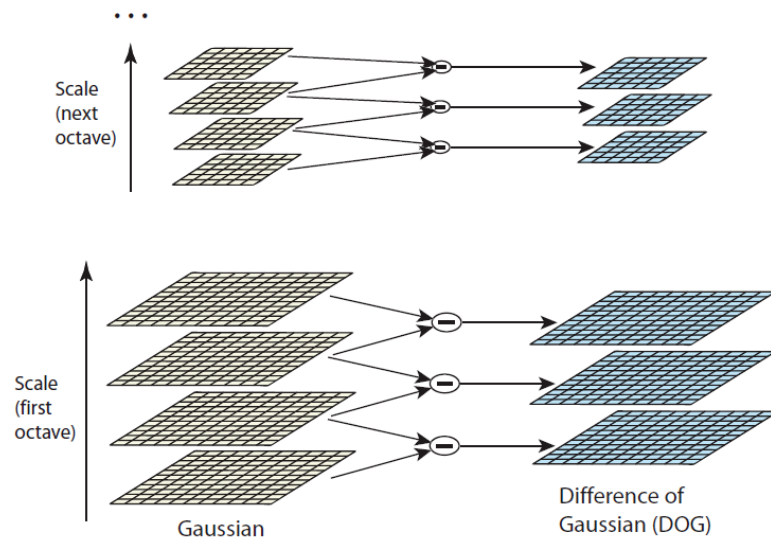
- Find keypoints (extrema) using DoG (approximated LoG)
- Idea: Find Corners, but scale invariance



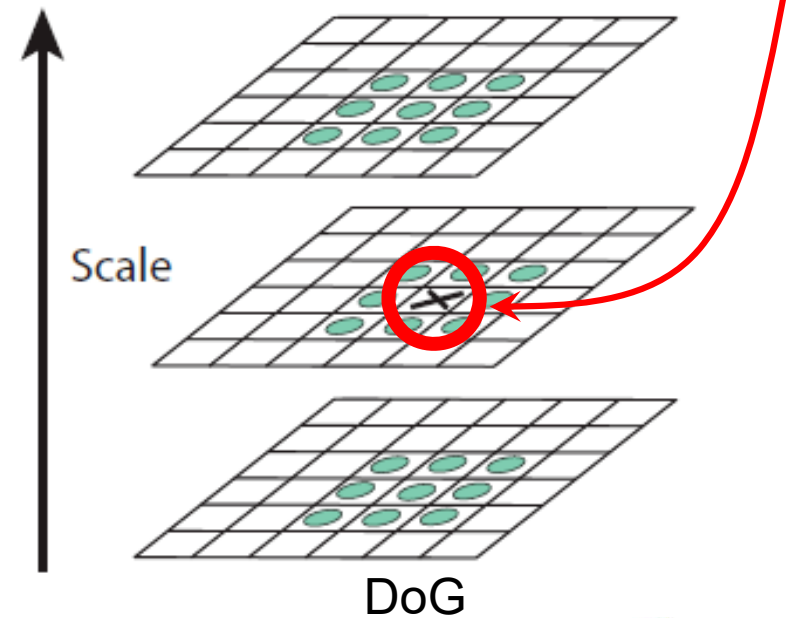
SIFT Features

- Keypoint detection

- If pixel x 's value is higher (or lower) than 26 pixels around it, the pixel x can be named as a keypoint



Maxima and minima
in DoG images



SIFT Features

1. **Enforce invariance to scale**: Compute Gaussian difference max, for many different scales; non-maximum suppression, find local maxima: keypoint candidates
2. **Localizable corner**: For each maximum fit quadratic function. Compute center with sub-pixel accuracy by setting first derivative to zero.
3. **Eliminate edges**: Compute ratio of eigenvalues, drop keypoints for which this ratio is larger than a threshold.
4. **Enforce invariance to orientation**: Compute orientation, to achieve rotation invariance, by finding the strongest second derivative direction in the smoothed image (possibly multiple orientations). Rotate patch so that orientation points up.
5. **Compute feature signature**: Compute a "gradient histogram" of the local image region in a 4x4 pixel region. Do this for 4x4 regions of that size. Orient so that largest gradient points up (possibly multiple solutions). Result: feature vector with 128 values (15 fields, 8 gradients).
6. **Enforce invariance to illumination change and camera saturation**: Normalize to unit length to increase invariance to illumination. Then threshold all gradients, to become invariant to camera saturation.

SIFT Features

- Keypoint detection

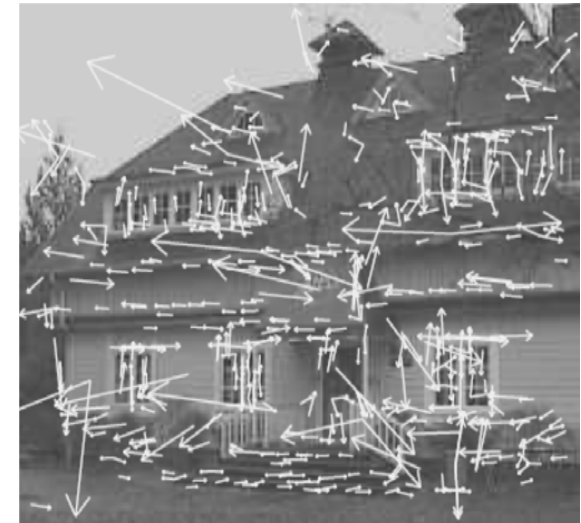
- Remove inappropriate keypoints (threshold)
- Eliminate the edge responses based on Harris response function
(Laplacian has strong response along edge)



233x189 image



extrema locations



extrema locations

(from 832 keypoints to 536 keypoints)

SIFT Features

1. **Enforce invariance to scale**: Compute Gaussian difference max, for many different scales; non-maximum suppression, find local maxima: keypoint candidates
2. **Localizable corner**: For each maximum fit quadratic function. Compute center with sub-pixel accuracy by setting first derivative to zero.
3. **Eliminate edges**: Compute ratio of eigenvalues, drop keypoints for which this ratio is larger than a threshold.
4. **Enforce invariance to orientation**: Compute orientation, to achieve rotation invariance, by finding the strongest second derivative direction in the smoothed image (possibly multiple orientations). Rotate patch so that orientation points up.
5. **Compute feature signature**: Compute a "gradient histogram" of the local image region in a 4x4 pixel region. Do this for 4x4 regions of that size. Orient so that largest gradient points up (possibly multiple solutions). Result: feature vector with 128 values (15 fields, 8 gradients).
6. **Enforce invariance to illumination change and camera saturation**: Normalize to unit length to increase invariance to illumination. Then threshold all gradients, to become invariant to camera saturation.

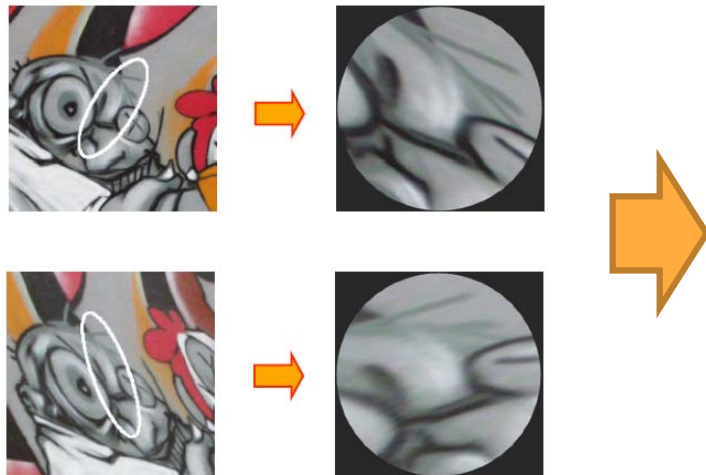
SIFT Features

- From covariant detection to invariant description
 - Scaled and rotated versions of the same neighborhood will give rise to blobs that are related by the same transformation
- What to do if we want to compare the appearance of these image regions?
 - *Normalization*: transform these regions into same-size circles rotated to a consistent reference orientation



SIFT Features

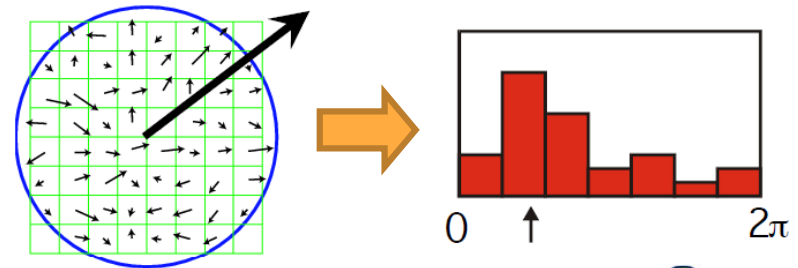
- From Blob detection problem (lec. 05)
 - There is no unique transformation from an ellipse to a unit circle



Select canonical orientation

Create histogram of local gradient directions computed at selected scale

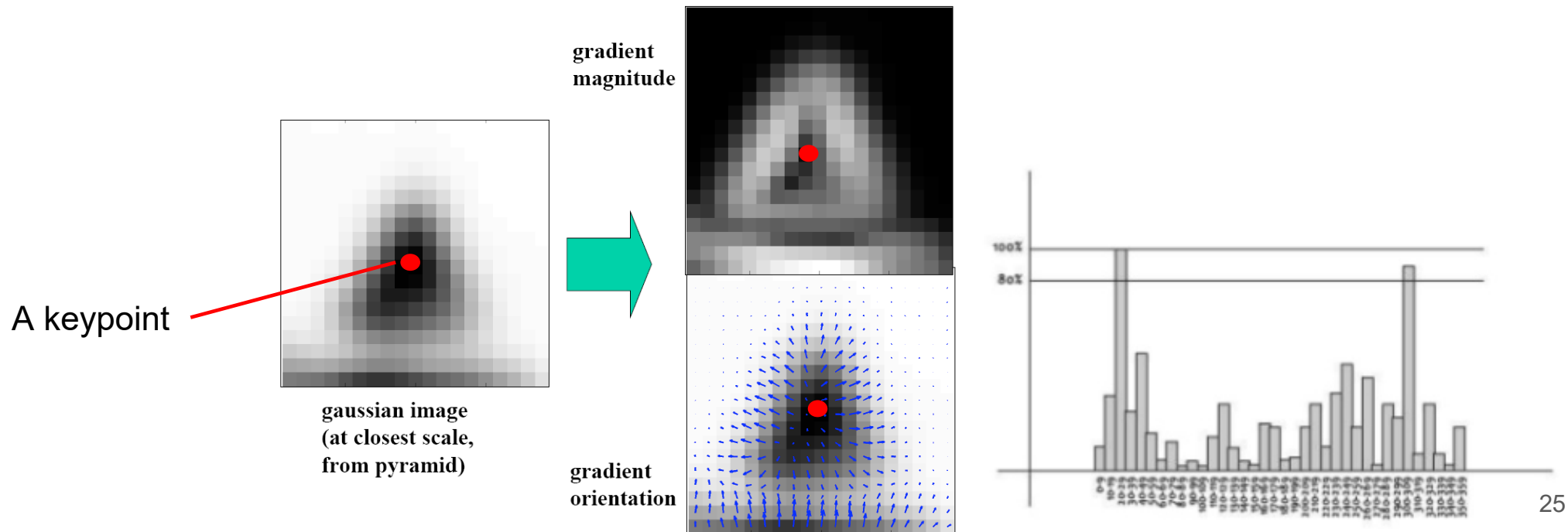
- ✓ Assign canonical orientation at peak of smoothed histogram
- ✓ Each key specifies stable 2D coordinates (x, y, scale, orientation)



SIFT Features

2) Keypoint description

- Create a histogram of local gradient directions at selected scale
- Orientation of a keypoint = the largest value of surrounding gradients



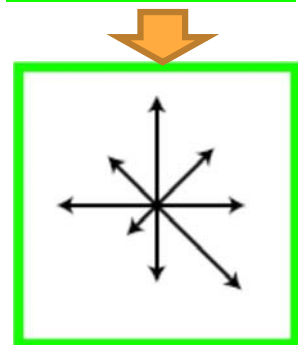
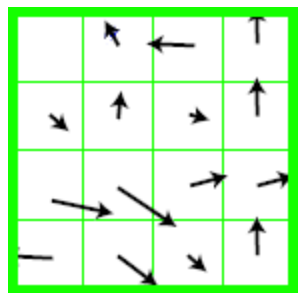
SIFT Features

1. **Enforce invariance to scale**: Compute Gaussian difference max, for many different scales; non-maximum suppression, find local maxima: keypoint candidates
2. **Localizable corner**: For each maximum fit quadratic function. Compute center with sub-pixel accuracy by setting first derivative to zero.
3. **Eliminate edges**: Compute ratio of eigenvalues, drop keypoints for which this ratio is larger than a threshold.
4. **Enforce invariance to orientation**: Compute orientation, to achieve rotation invariance, by finding the strongest second derivative direction in the smoothed image (possibly multiple orientations). Rotate patch so that orientation points up.
5. **Compute feature signature**: Compute a "gradient histogram" of the local image region in a 4x4 pixel region. Do this for 4x4 regions of that size. Orient so that largest gradient points up (possibly multiple solutions). Result: feature vector with 128 values (15 fields, 8 gradients).
6. **Enforce invariance to illumination change and camera saturation**: Normalize to unit length to increase invariance to illumination. Then threshold all gradients, to become invariant to camera saturation.

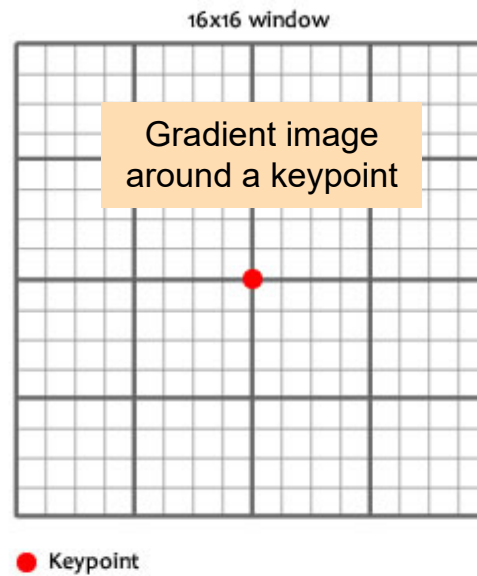
SIFT Features

■ Keypoint description

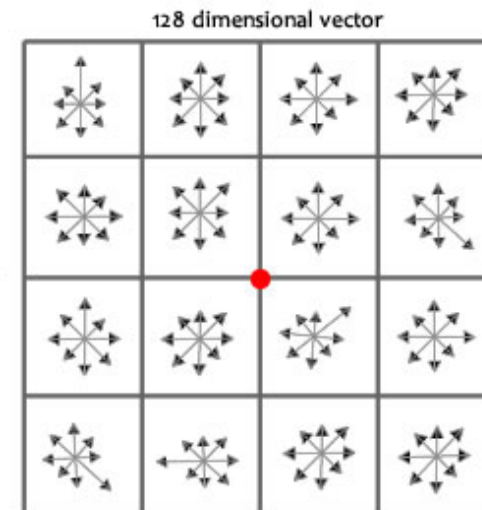
- Compute a descriptor for the local image region about each keypoint
- Keypoint (x, y, scale, orientation) = [feature vector]



8 bin orientation



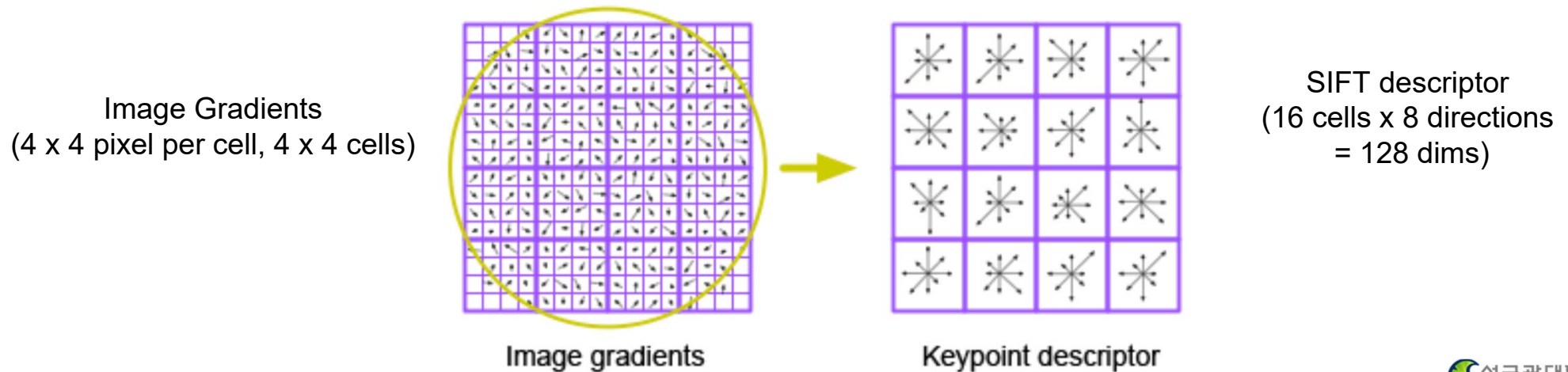
● Keypoint



128 dimensional vector

SIFT Features

- Keypoint description -> SIFT vector formation
 - Thresholded image gradients are sampled over 16x16 array of locations in scale space
 - Create array of orientation histograms
 - 8 orientations x 4x4 histogram array = 128 dimensions
 - Keypoint : (x, y, scale, orientation, feature vector)



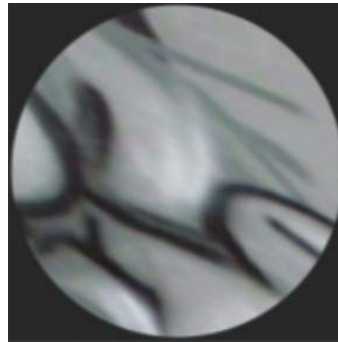
SIFT Features

■ Descriptors

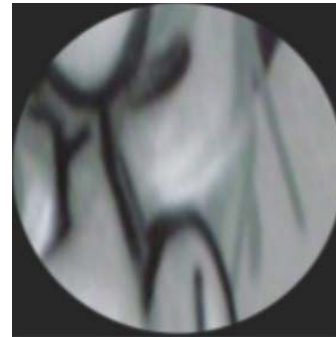
Extract affine regions



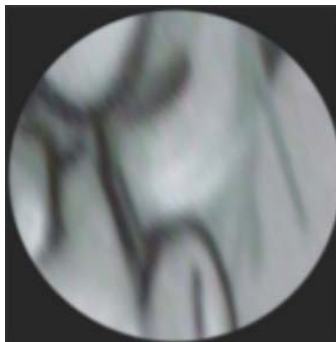
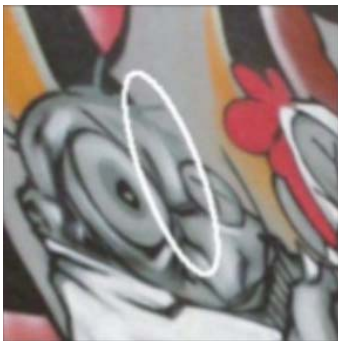
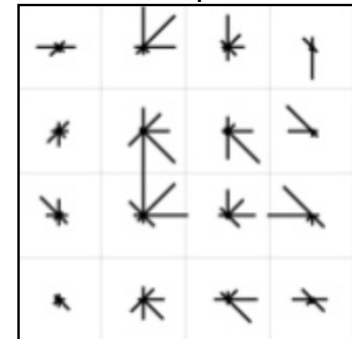
Normalize regions



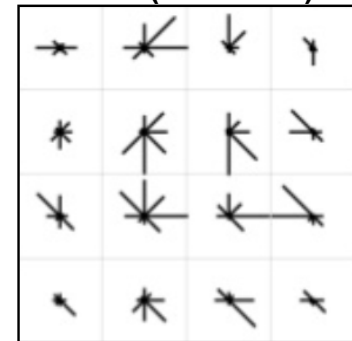
Eliminate rotational
+illumination



Compute appearance
descriptors

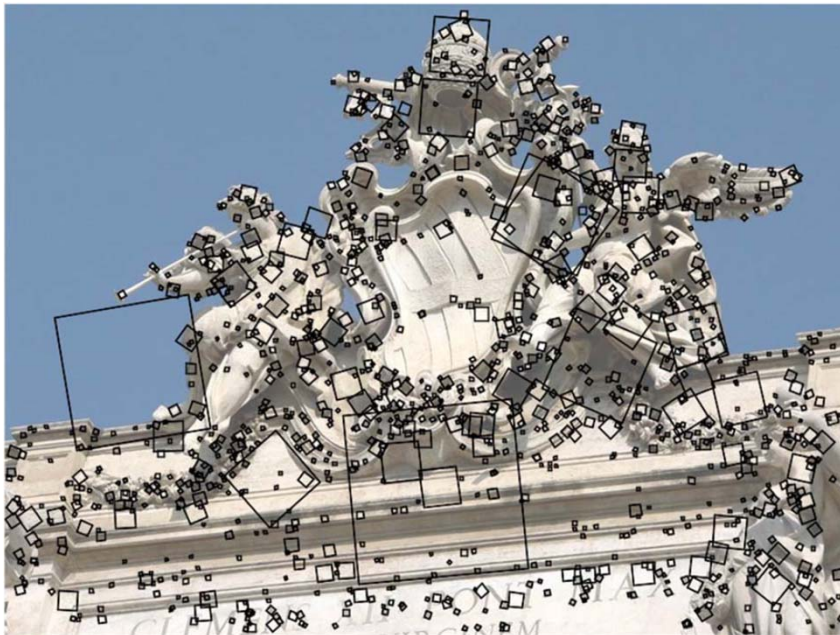


SIFT (Lowe '04)



SIFT Features

- Keypoint description
 - Detected features with characteristic scales and orientations



SIFT Features

- Keypoint description
 - Detected features with characteristic scales and orientations



SIFT Features

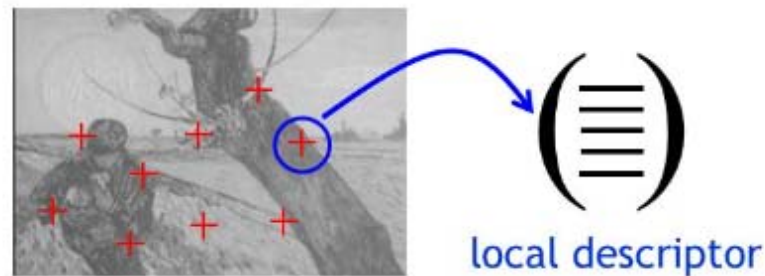
■ SIFT stages:

- Scale-space extrema detection
- Keypoint localization

detector

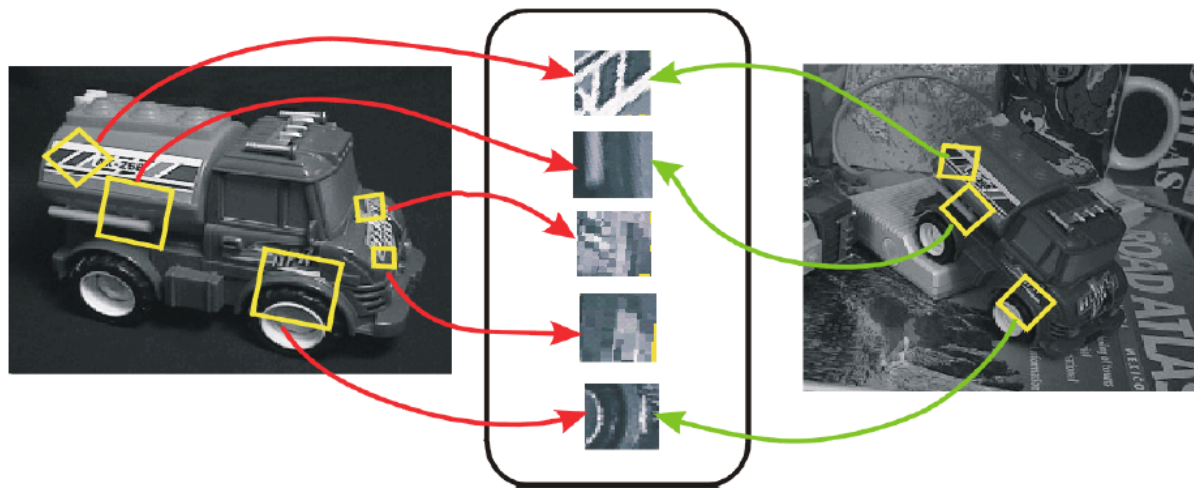
- Orientation assignment
- Keypoint descriptor

descriptor



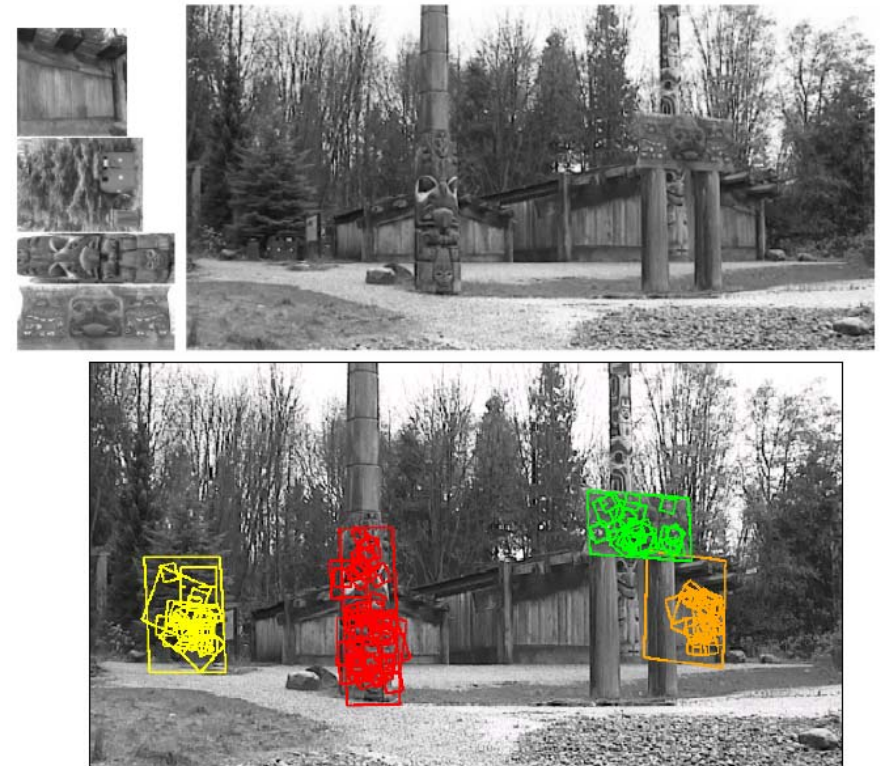
SIFT Features

- Detection is covariant:
 - $\text{features}(\text{transform}(\text{image})) = \text{transform}(\text{features}(\text{image}))$
- Description is invariant:
 - $\text{features}(\text{transform}(\text{image})) = \text{features}(\text{image})$



SIFT Features

- Extraordinarily robust detection and description technique
 - Can handle changes in viewpoint
 - Up to about 60 degree out-of-plane rotation
 - Can handle significant changes in illumination
 - Sometimes even day vs. night
 - Fast and efficient — can run in real time
 - Lots of code available



https://docs.opencv.org/master/da/df5/tutorial_py_sift_intro.html

SURF

SURF = Speeded-Up Robust Features

SURF: Speeded Up Robust Features.

Herbert Bay, Tinne Tuytelaars, and Luc Van Gool, ECCV 2006.

SURF

■ Aim

- SIFT was comparatively slow and people needed more speeded-up version
- A new algorithm called SURF introduced, it is a speeded-up version of SIFT
- 3~7 times faster than SIFT, with similar matching performance

■ Key Idea

- Use Haar filters and Integral Images
- It is **good** handling images with **blurring and rotation**, but **not good** at handling **viewpoint change and illumination change**

SURF

- Algorithm

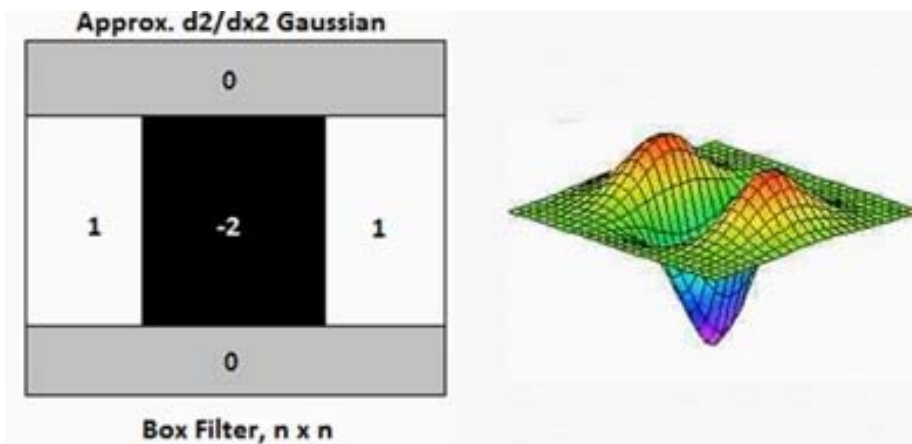
- This algorithm uses Hessian - Laplacian to approximate Laplacian of Gaussian, to improve calculation speed

- Two-step Algorithm

- Interest point detection through filtering/convolution
- Feature classification using descriptor vector (Haar wavelets)

SURF

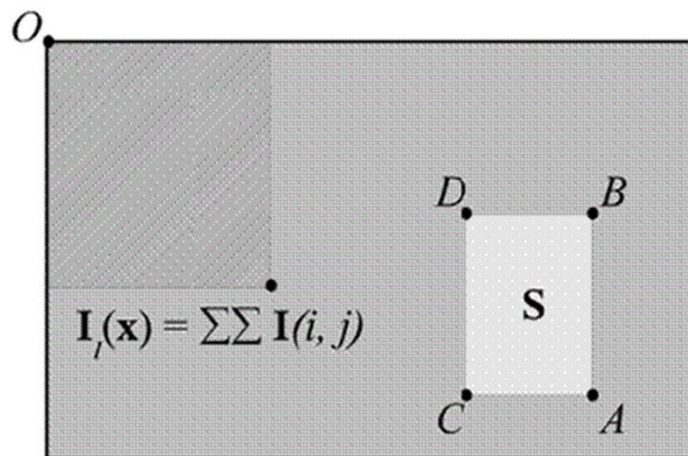
- Interest point detection
 - Approximating the determinant of the Hessian matrix
 - If determinant is local max $\rightarrow$ Interest point
 - Hessian approx. using Integral image and Box filters



SURF

- Integral image

- Using integral images for major speed up
- Integral Image (summed area tables) is an intermediate representation for the image and contains the sum of gray scale pixel values of image



$$S = A - B - C + D$$

SURF Features

■ Hessian Matrix

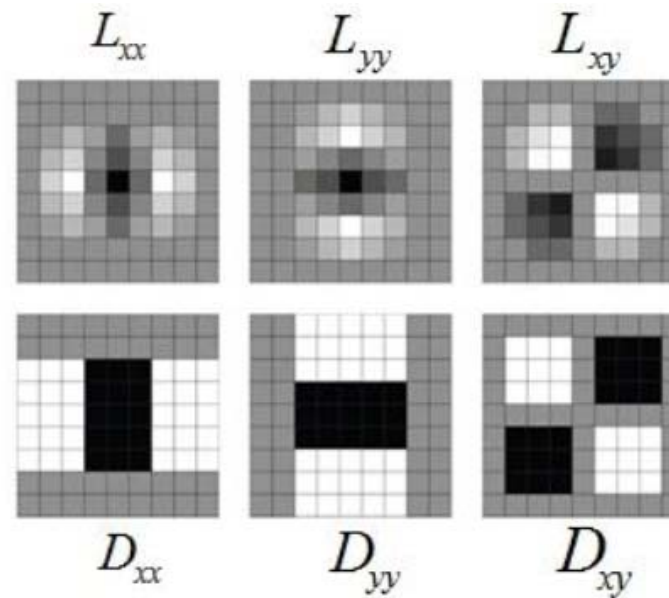
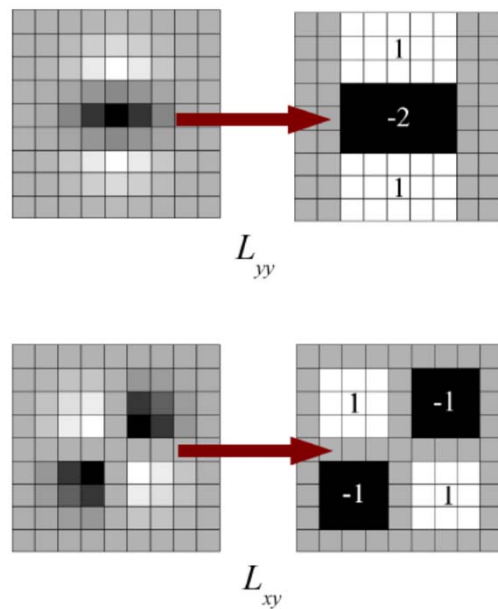
- It approximates LoG with Box Filter
- Convolution with box filter can be easily calculated with the help of integral images
- Determinant of Hessian matrix for both scale and location.

$$\mathcal{H}(\mathbf{x}, \sigma) = \begin{bmatrix} L_{xx}(\mathbf{x}, \sigma) & L_{xy}(\mathbf{x}, \sigma) \\ L_{xy}(\mathbf{x}, \sigma) & L_{yy}(\mathbf{x}, \sigma) \end{bmatrix}$$

$$\det(\mathcal{H}_{\text{approx}}) = D_{xx}D_{yy} - (wD_{xy})^2.$$

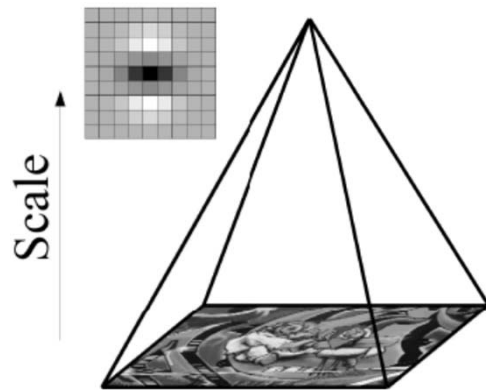
SURF

- Approximated second order derivatives with box filters
 - Convolution with box filter can be easily calculated with the help of integral images

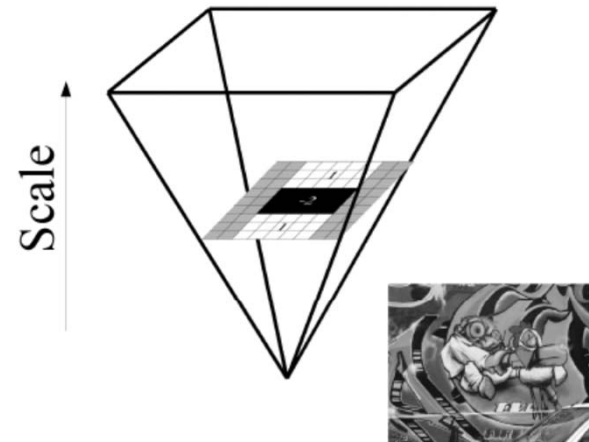


SURF

- Scale analysis with constant image size
 - Instead of iteratively reducing the image size , the use of integral images allows the up-scaling of the filter at constant cost



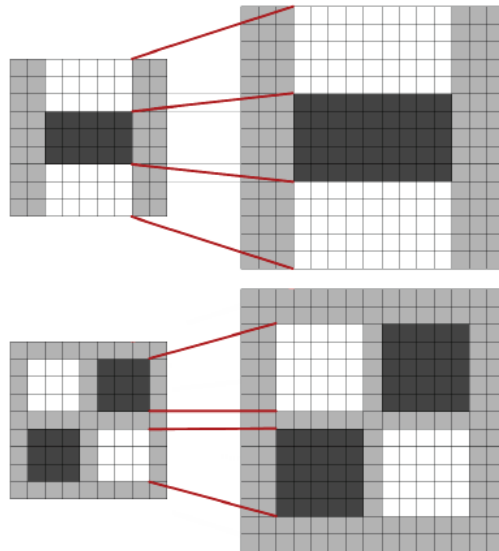
SIFT



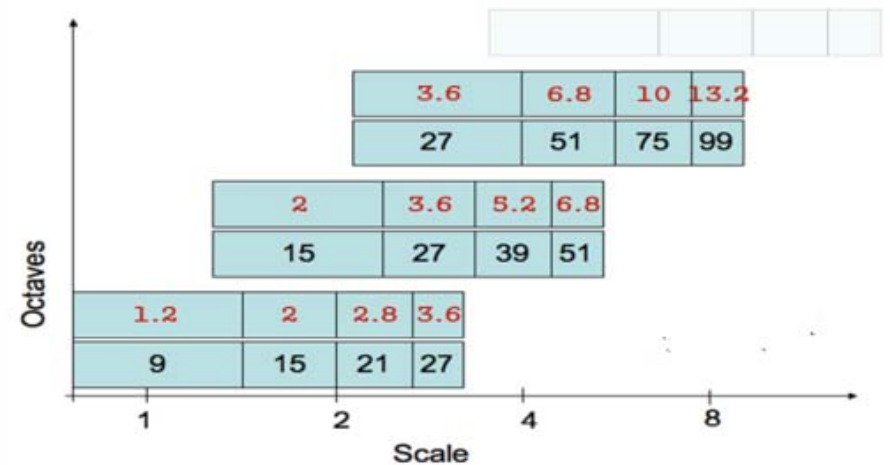
SURF

SURF

- Scale analysis with constant image size



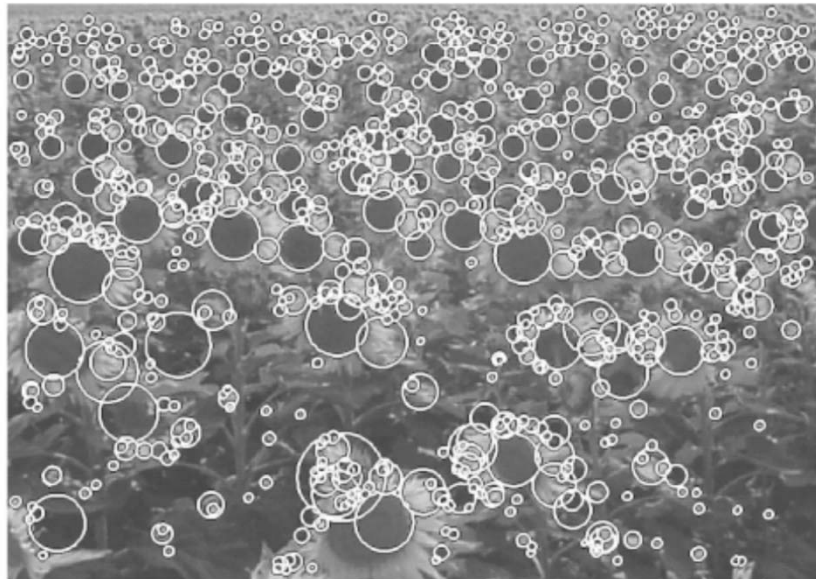
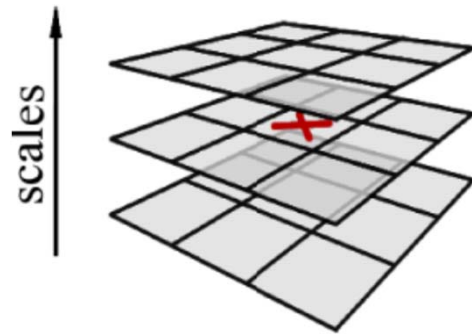
Filters D_{yy} (top) and D_{xy} (bottom) for two successive scale levels (9x9 and 15x15)



Graphical representation of the filter side lengths for three different octaves

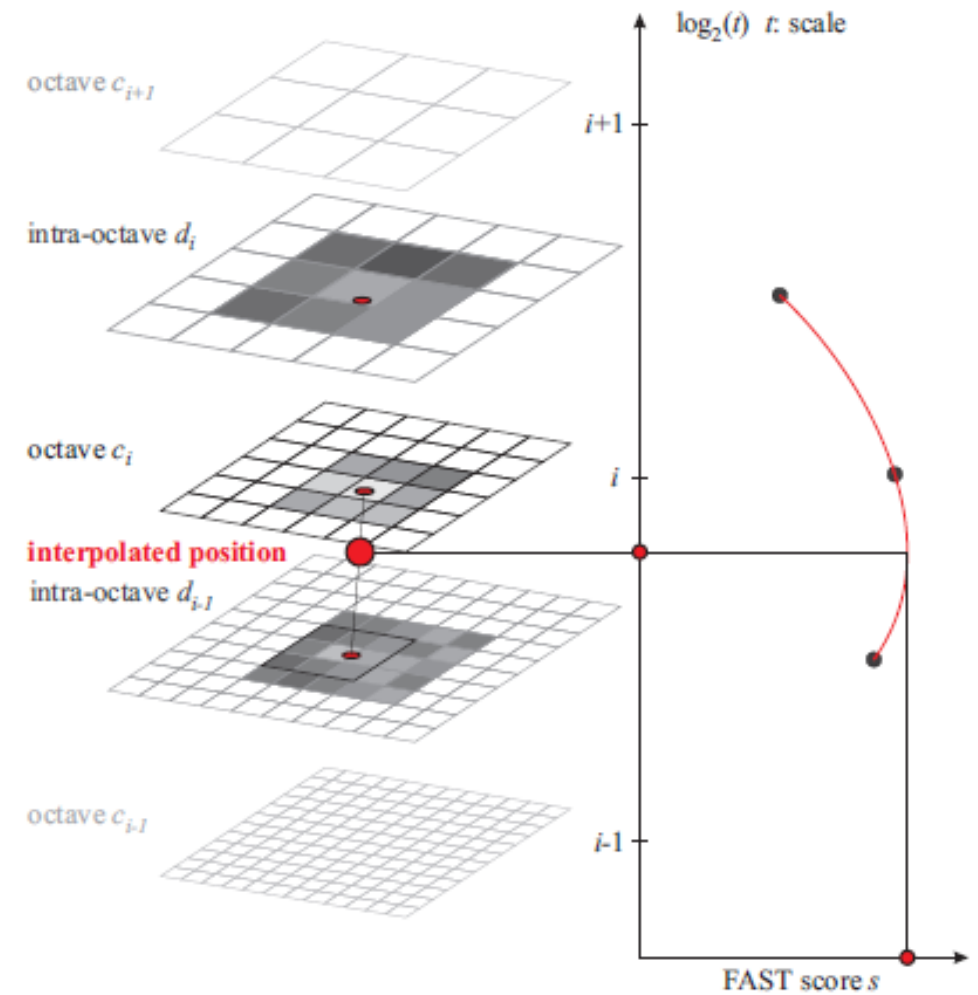
SURF

- Non-maximum suppression and interpolation
 - Blob-like feature detector
 - Same as SIFT



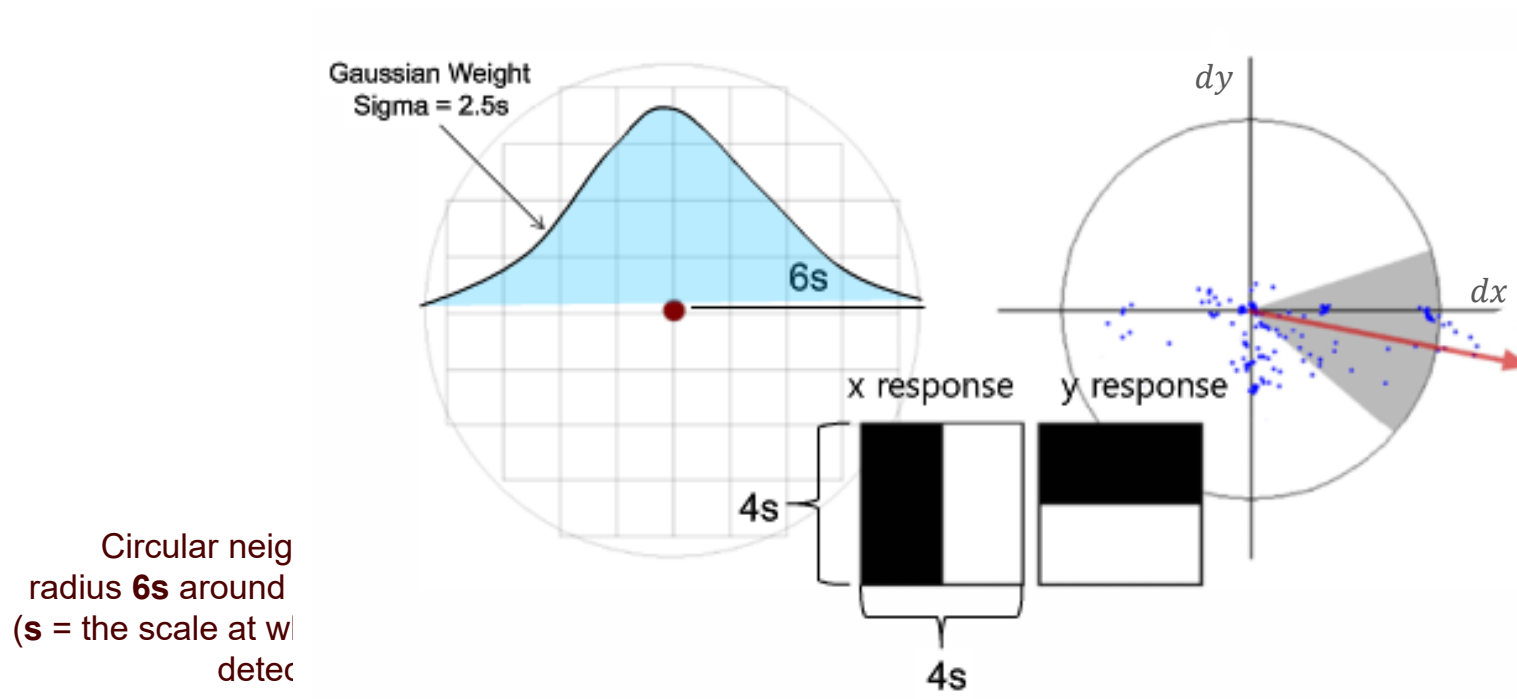
SURF

- Keypoint Localization
 - Select the interpolated position



SURF

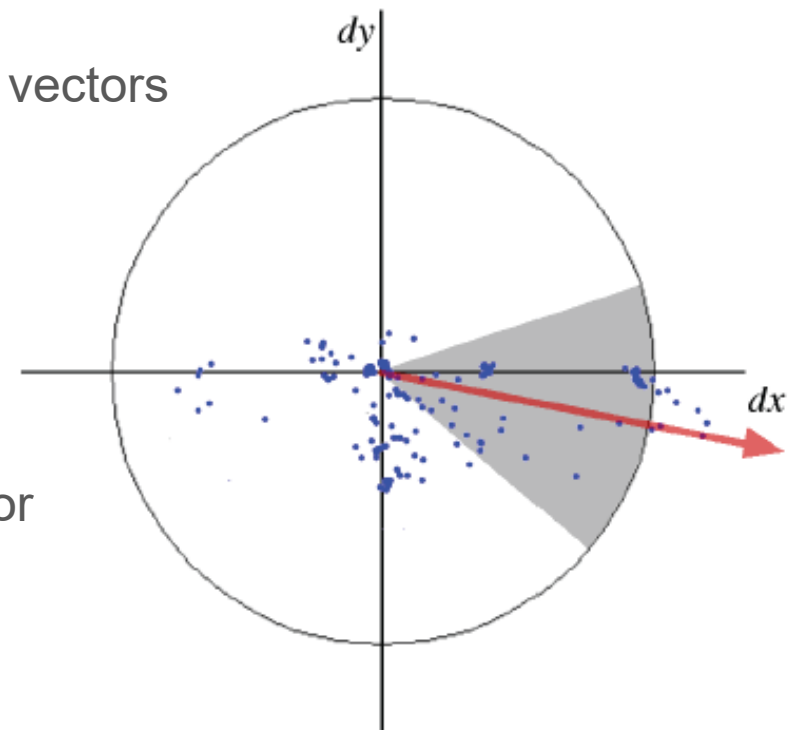
- Orientation Assignment



SURF

■ Dominant orientation

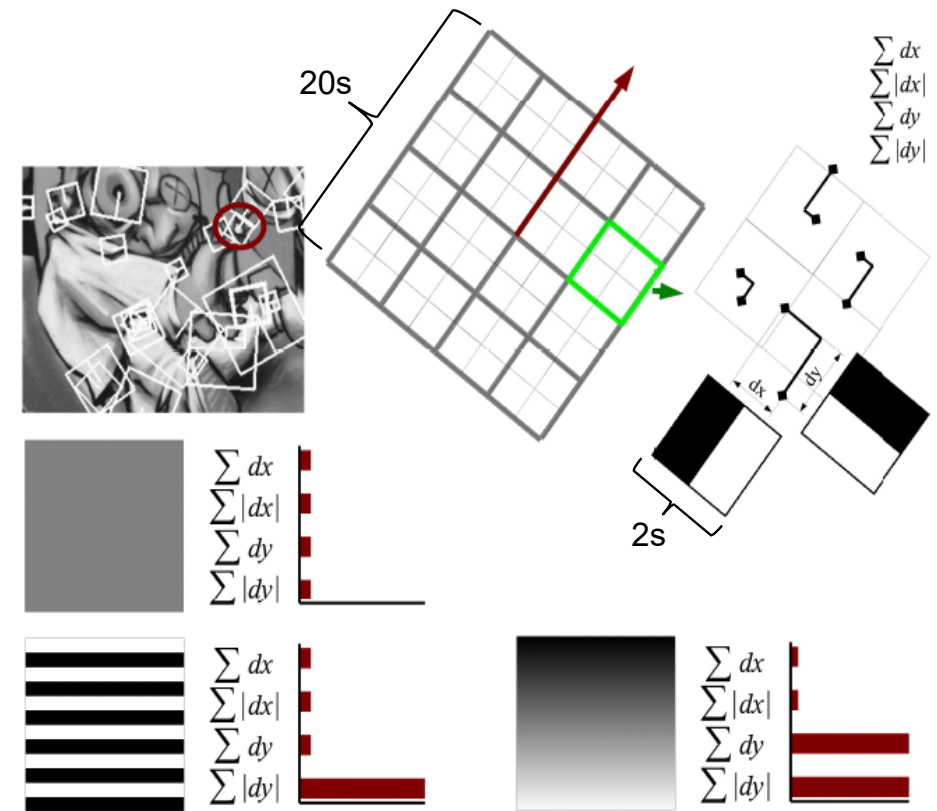
- The *Haar wavelet* responses are represented as vectors
- **Sum all responses** within a sliding orientation window covering an angle of 60 degree
- The x, y two summed response yield a new vector
- The longest vector is the dominant orientation



SURF

■ Description

- Split the interest region (20s x 20s) up into 4x4 square sub-regions
- Calculate Haar wavelet response d_x and d_y and weight the response with a Gaussian kernel
- Sum the response over each sub-region for d_x and d_y , then sum the absolute value of response
- Normalize the vector into unit length

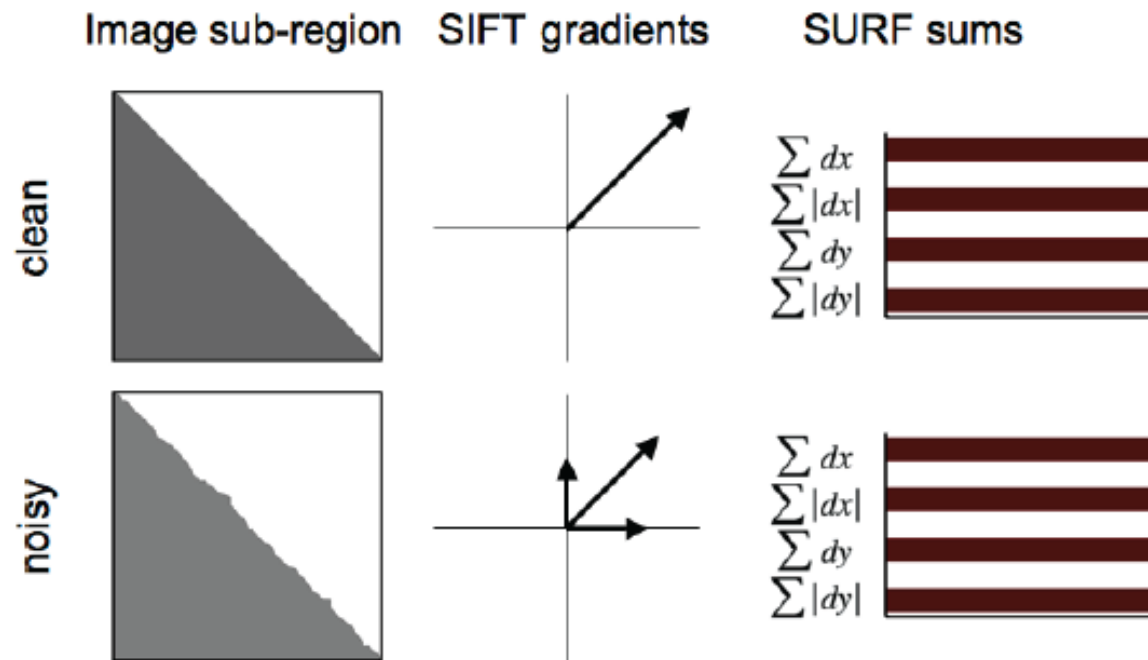


SURF

https://docs.opencv.org/master/df/dd2/tutorial_py_surf_intro.html

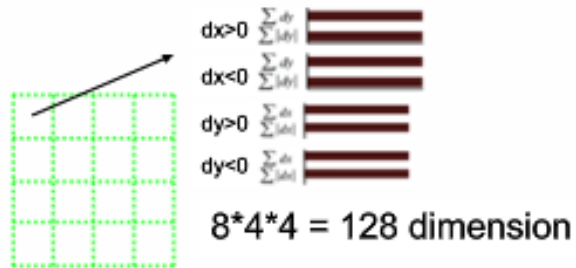
■ Analysis and Conclusion

- SURF is **faster** than SIFT by **3 times**, and has recall precision not worse than SIFT

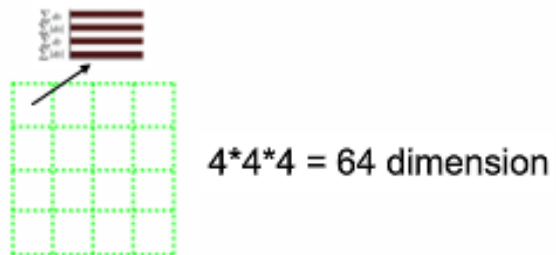


SURF

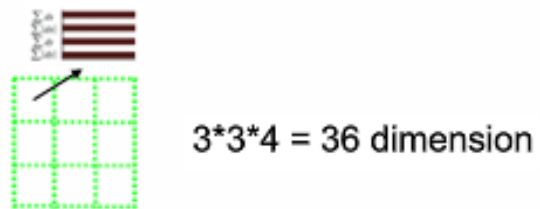
■ Analysis and Conclusion



- SURF-128
 - Compute : not slower
 - Matching : slower



- SURF-64
 - best result

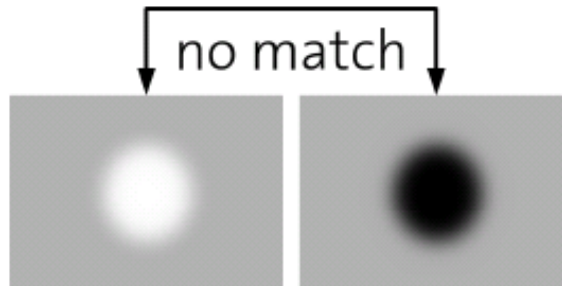


- SURF-36
 - Accuracy : slightly worse
 - Matching : very fast

SURF

- Matching - Fast indexing
 - through the sign of the Laplacian for the underlying interest point
 - The sign of trace of the Hessian matrix

$$Trace = L_{xx} + L_{yy}$$



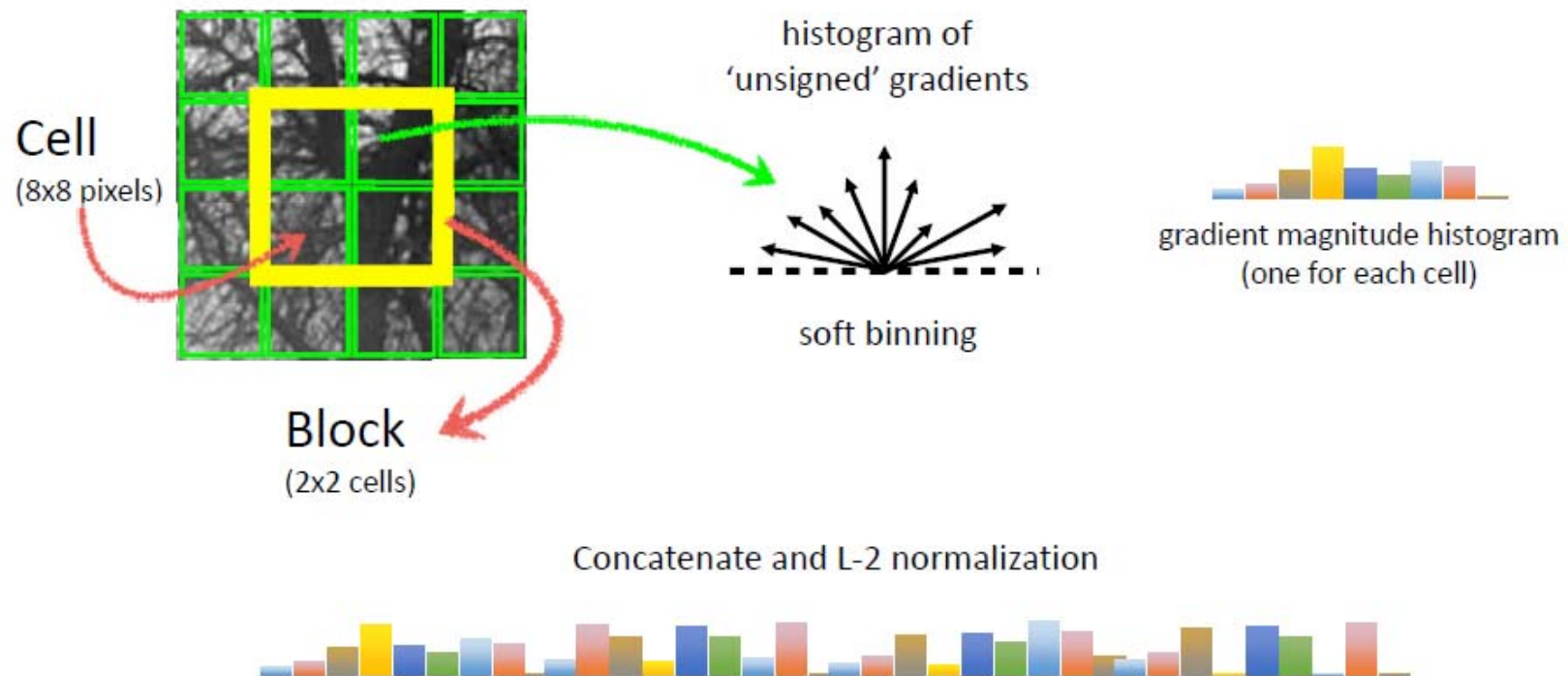
HOG

HOG = Histogram of Oriented Gradient

Histograms of Oriented Gradients for Human Detection.
Navneet Dalal and Bill Triggs, CVPR 2005.

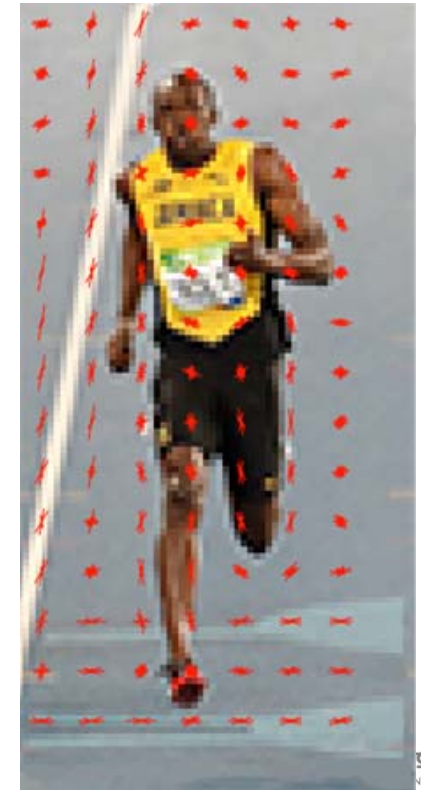
HOG features

- HOG (Histogram of Oriented Gradient)



HOG features

- HOG (Histogram of Oriented Gradient)
 - 1) Gradient computation
 - 2) Calculate Histogram of Gradients in 8×8 cells
 - 3) Block normalization
 - 4) Calculate the HOG feature vector



HOG features

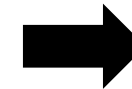
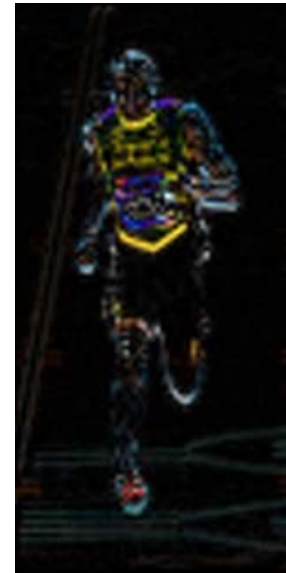
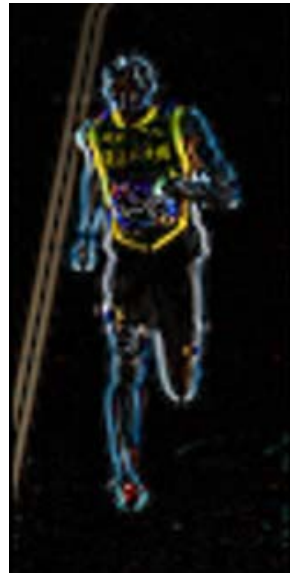
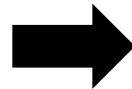
- 1) Gradient computation

X-gradient (g_x)

-1	0	1
----	---	---

Y-gradient (g_y)

-1
0
1



Gradient direction

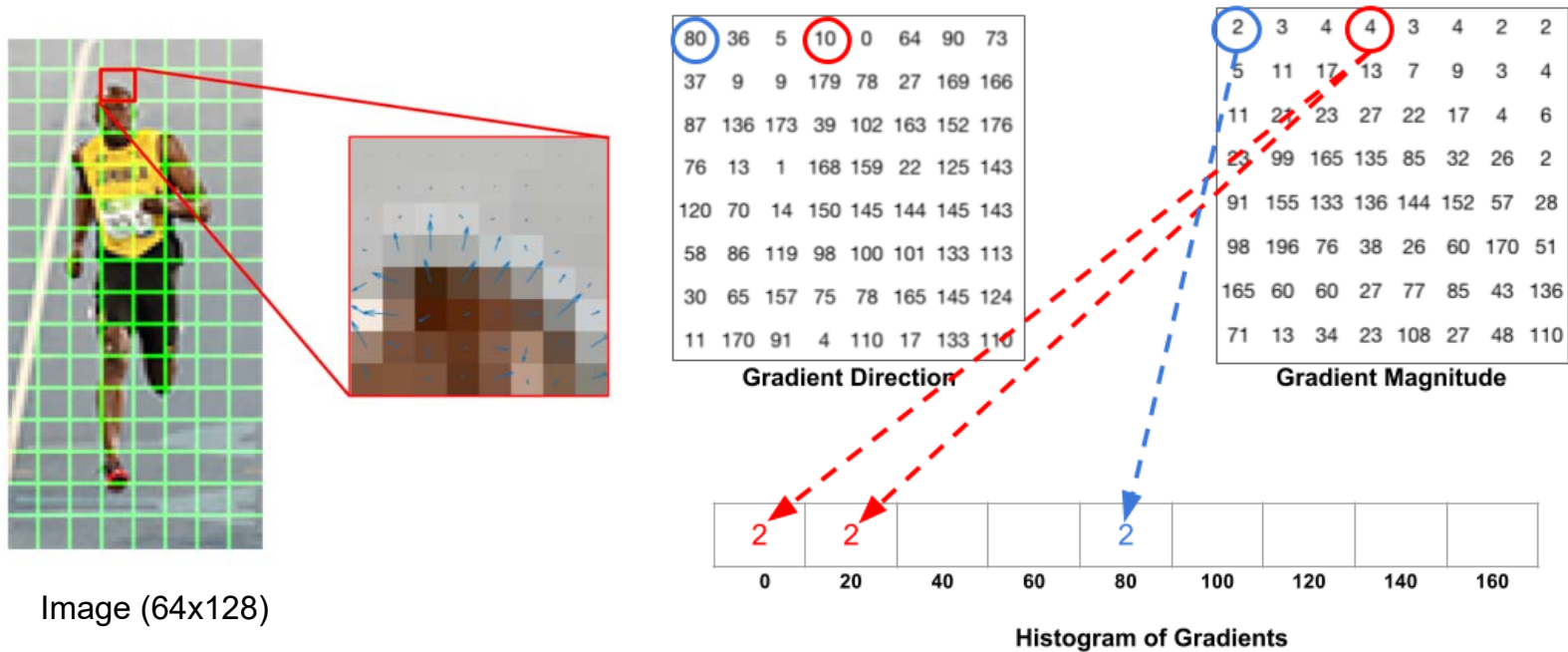
$$g = \arctan \frac{g_y}{g_x}$$

Gradient magnitude

$$g = \sqrt{g_x^2 + g_y^2}$$

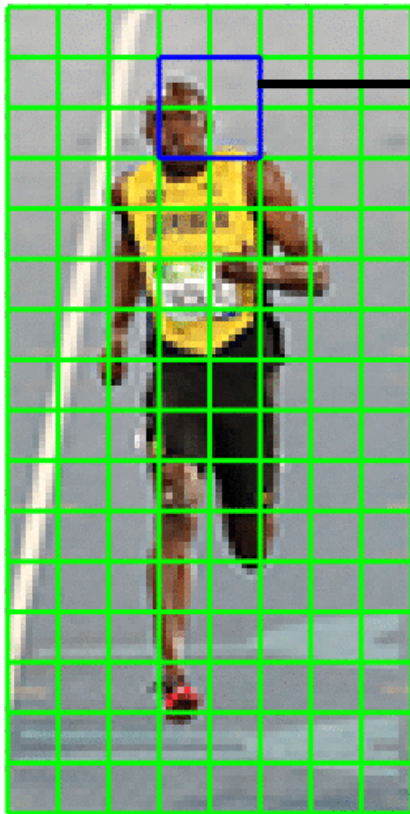
HOG features

- 2) Calculate Histogram of Gradients in 8×8 cells



HOG features

■ 3) Block normalization (contrast normalization)



Cell1	v00	v01	v02	...	v09
Cell2	v10	v11	v12	...	v19
Cell3	v20	v21	v22	...	v29
Cell4	v30	v31	v32	...	v39

Histogram of gradients

→ Concatenate 4 * 9 and normalize by $\frac{1}{\sum v_{ij}}$

→ 1 block vector = 4 cells * 9 normalized elements

HOG features

- 4) Calculate the HOG feature vector

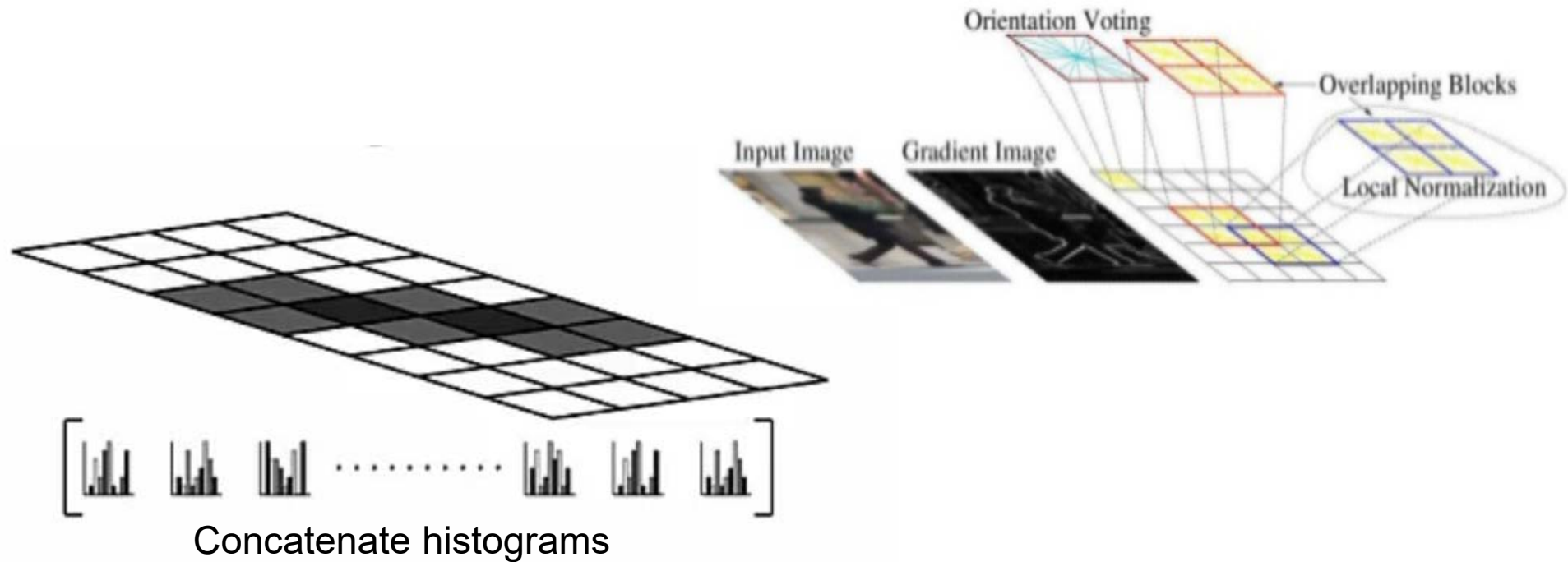


Image (64x128)

- Total 105 blocks (7x15, overlapping)
- 105 blocks * 4 cells * 9 normalized elements
- 3780-sized feature vector

HOG features

- Make it a 1D vector of length 3780



HOG features

- 4) Calculate the HOG feature vector

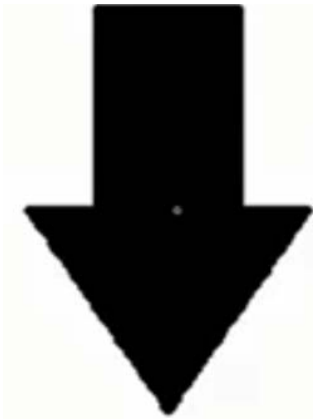


← Visualized by averaging the 4 normalized cells

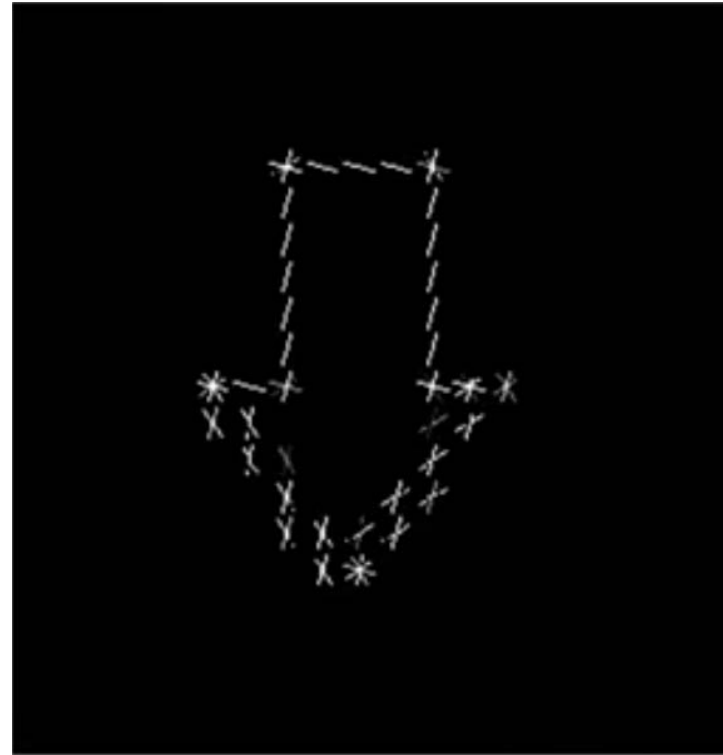
HOG features

https://docs.opencv.org/master/dd/d3b/tutorial_py_svm_opencv.html

- Example



Input image



Histogram of Oriented Gradients

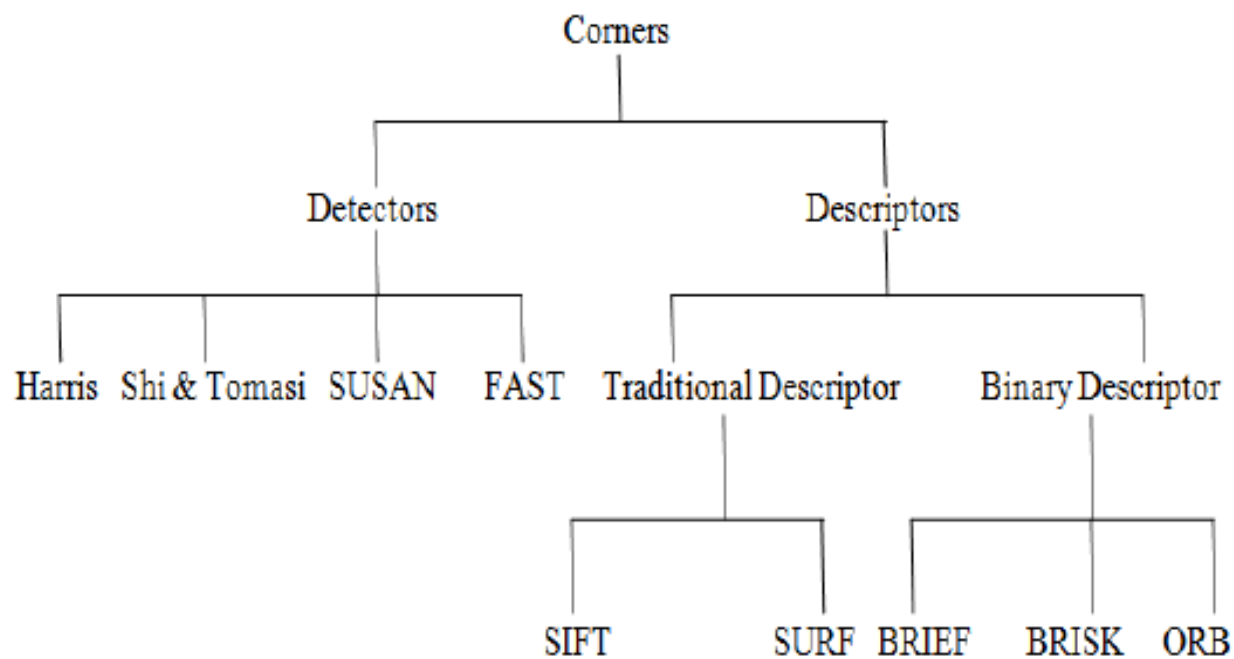


Figure 3: Classification of popular Corner Detectors and Descriptors

Source:

Uchida, Yusuke. "Local Feature Detectors, Descriptors, and Image Representations: A Survey." *arXiv preprint arXiv:1607.08368* (2016).

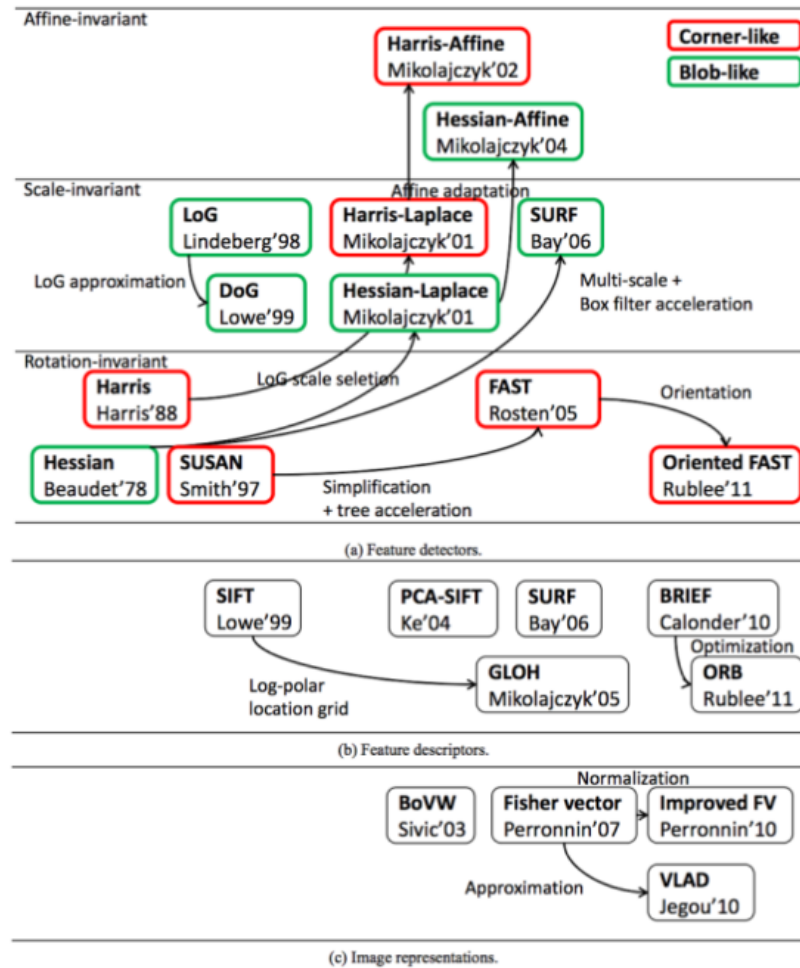


Figure 2. The overview of the history of local features and image representations. Only an especially important part of local features and image representations is shown.

BRIEF

BRIEF = Binary Robust Independent Elementary Features

BRIEF: Binary Robust Independent Elementary Features

Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua,
ECCV 2010. (also PAMI'12)

BRIEF

- Binary Robust Independent Elementary Features

- we define test on patch p of size $S \times S$ as

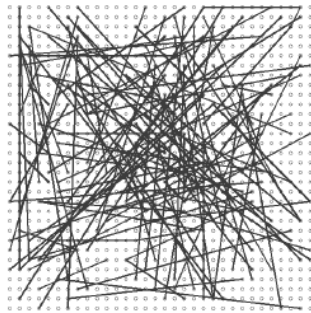
- Binary test
$$\tau(\mathbf{p}; \mathbf{x}, \mathbf{y}) := \begin{cases} 1 & \text{if } \mathbf{p}(\mathbf{x}) < \mathbf{p}(\mathbf{y}) \\ 0 & \text{otherwise} \end{cases}$$

- BRIEF descriptor
$$f_{n_d}(\mathbf{p}) := \sum_{1 \leq i \leq n_d} 2^{i-1} \tau(\mathbf{p}; \mathbf{x}_i, \mathbf{y}_i)$$

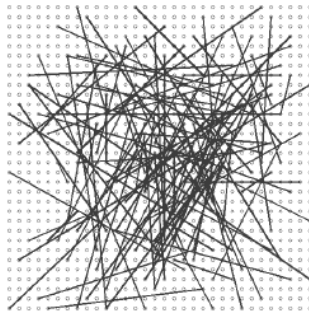
- For each $S \times S$ patch
 - Smooth it
 - Then pick pixels using pre-defined binary tests

BRIEF

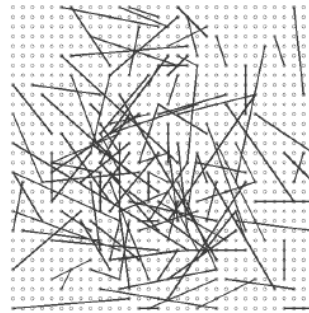
- Binary Robust Independent Elementary Features
 - Random selection of pairs of intensity values
 - Fixed sampling pattern of 128, 256, 512 pairs
 - Hamming distance to compare descriptors (XOR)



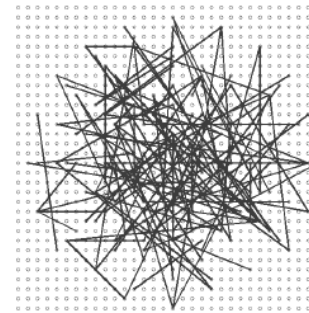
G I



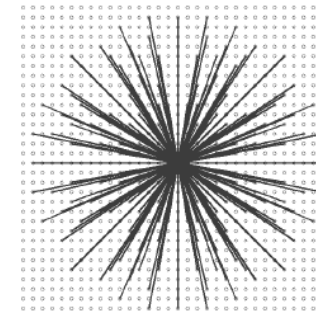
G II



G III



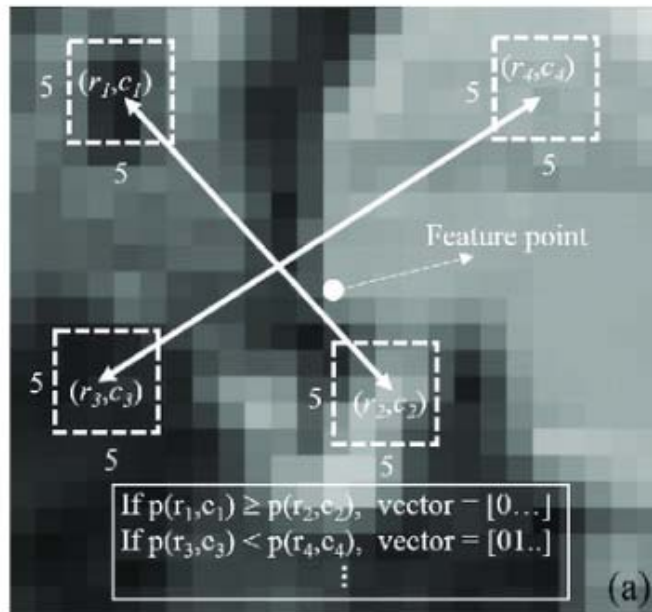
G IV



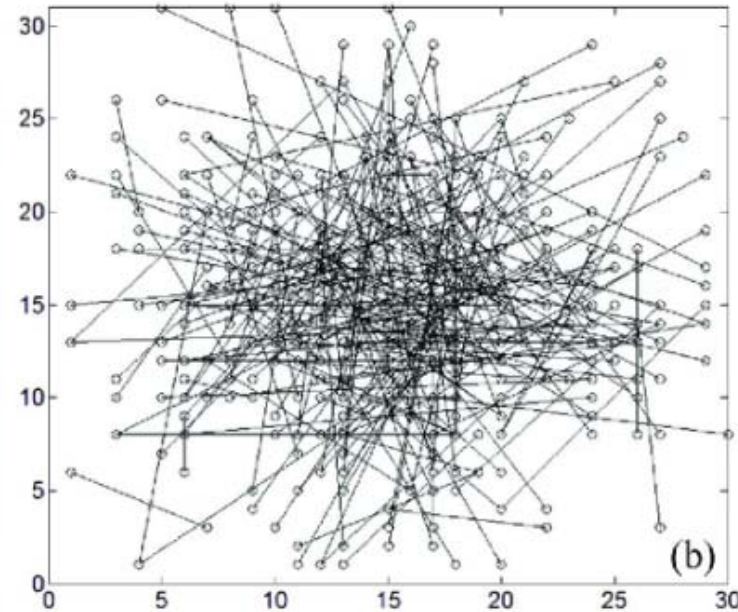
G V

BRIEF

- Binary Robust Independent Elementary Features



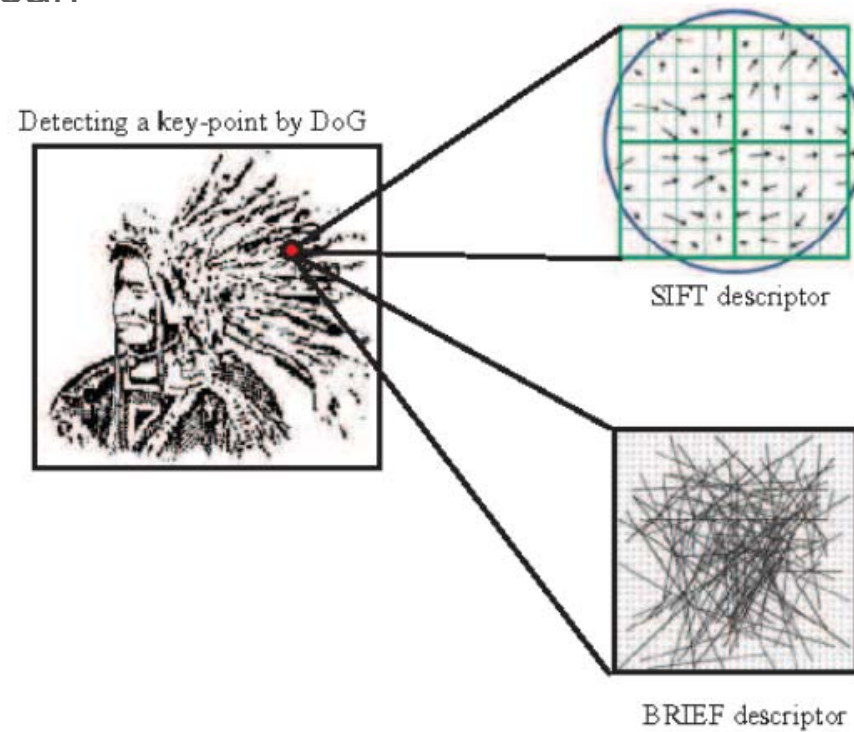
(a) Operation processing of the BRIEF descriptor



(b) Its 256 pixel-pairs

BRIEF

- Binary Robust Independent Scalable Keypoints
 - distance instead of Euclidean



BRIEF

- Pros:

- Compact, easy-computed, highly discriminative
- Fast matching using Hamming distance
- Good recognition performance

- Cons:

- More sensitive to image distortions and transformations, in particular to in-plane rotation and scale change

BRIEF

- Binary Robust Independent Scalable Keypoints
 - Combination of SIFT-like scale-space keypoint detection and BRIEF-like descriptor
 - BRISK is a 512 bit binary descriptor that computes the weighted Gaussian average over a select pattern of points near the keypoint;
 - It compares the values of specific pairs of Gaussian windows, leading to either 1 or 0, depending on which window in the pair was greater
 - The pairs to use are preselected in BRISK, This creates binary descriptors that work with hamming distance instead of Euclidean

BRIEF

- Correct matches



ORB

ORB: an efficient alternative to SIFT or SURF

ORB: an efficient alternative to SIFT or SURF.

Rublee, E., Rabaud, V., Konolige, K., and Bradski, G, ICCV 2011

ORB

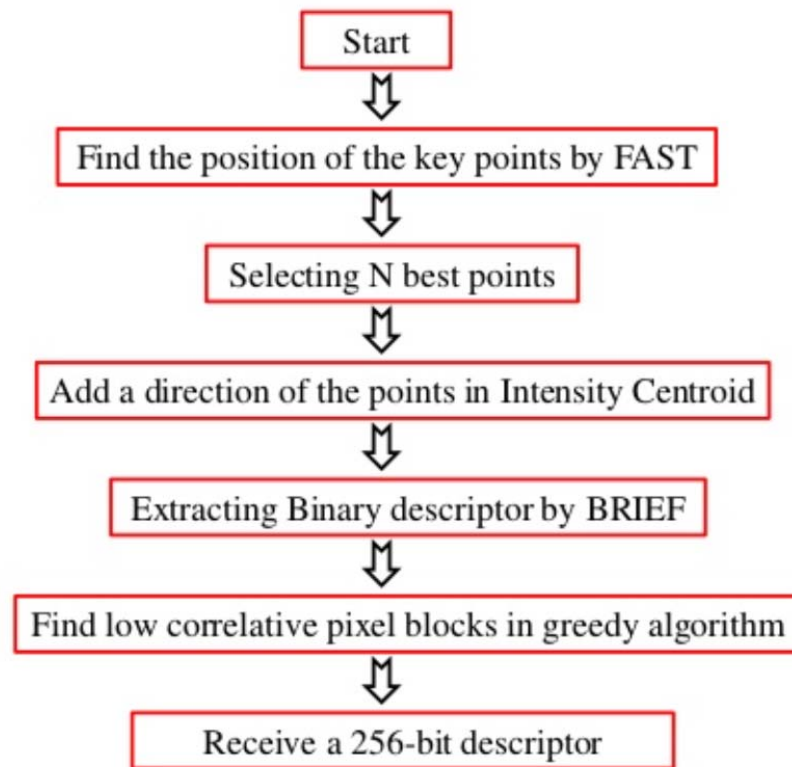
- Oriented FAST and Rotated BRIEF
 - Fast and accurate orientation compensation to FAST
 - Efficient computation of oriented BRIEF
 - Learn to de-correlate BRIEF in sampling pairs under rotational invariance
- Intensity centroid for corner orientation:

$$m_{pq} = \sum_{x,y} x^p y^q I(x, y) \quad C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right) \quad \theta = \text{atan2}(m_{01}, m_{10})$$



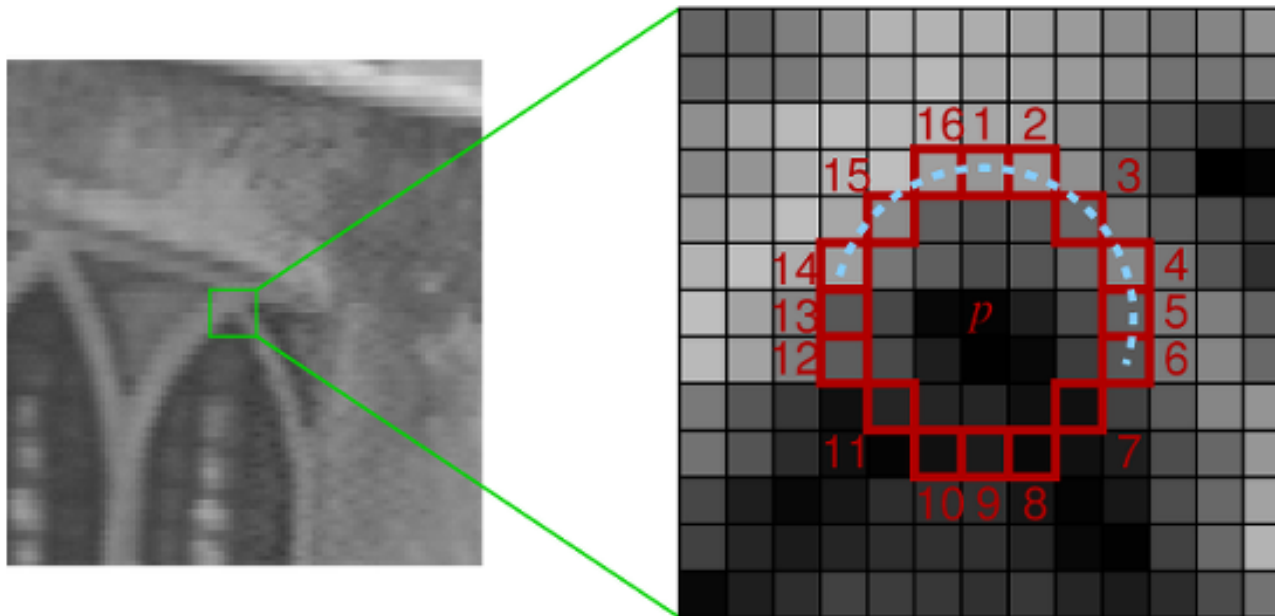
ORB

- The flow chart of basic ORB algorithm



ORB

- FAST corner detector

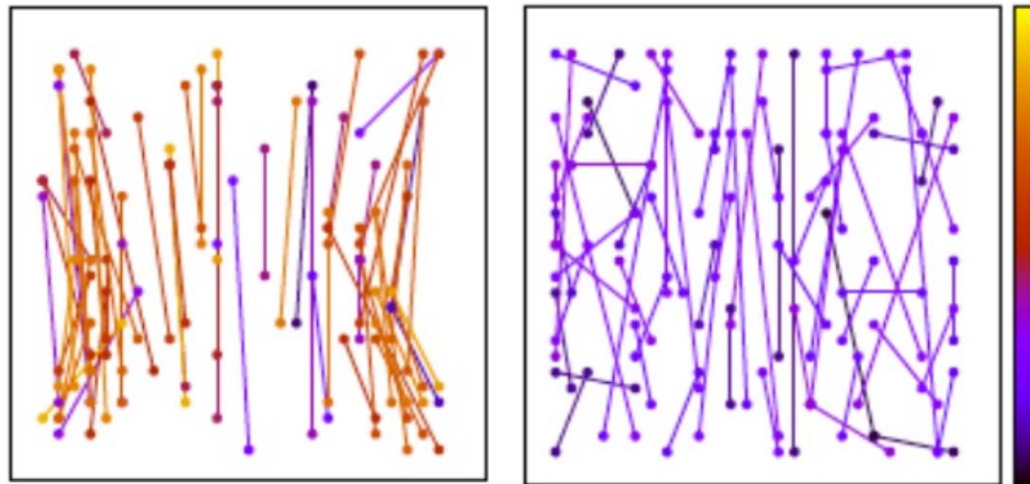


ORB

- Properties of the sampling pairs
 - Uncorrelation – each new pair will bring new information to the descriptor
 - High variance – more discriminative, since it responds differently to inputs
- Learning the sampling: a training set of about 300,000 keypoints drawn
 - Greedy method: obtain a set of 256 uncorrelated binary tests with high variance

ORB

- Select a good set of pairwise comparisons
 - Minimize correlation under various orientation changes



High variance

High variance + uncorrelated

ORB

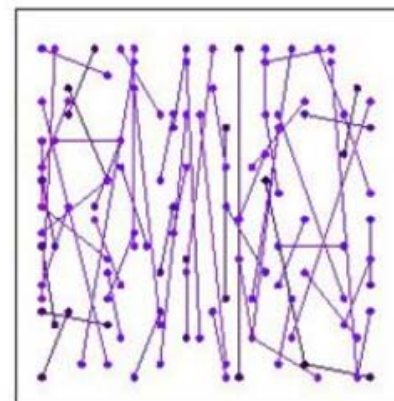
- It is robust to rotation than BRIEF

BRIEF



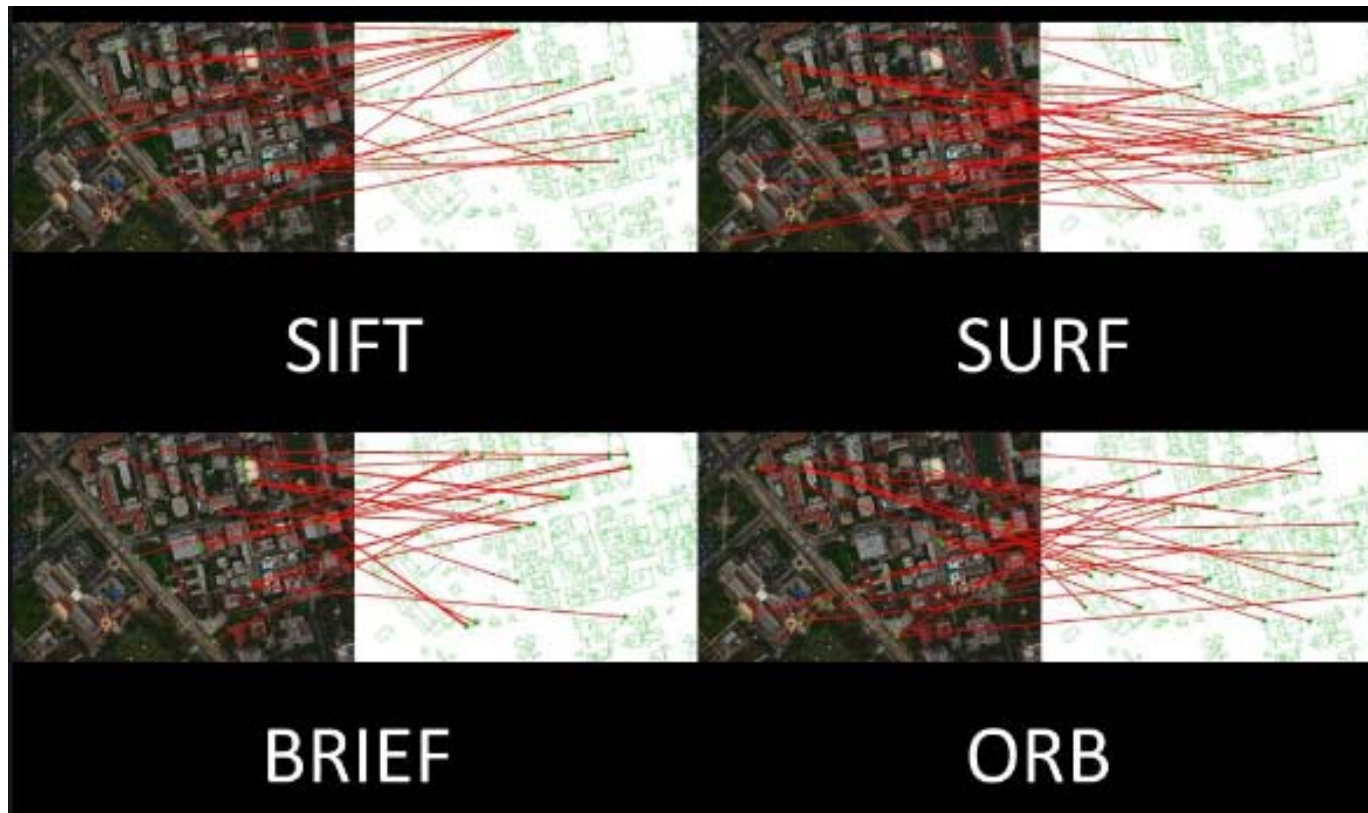
- Efficient

ORB



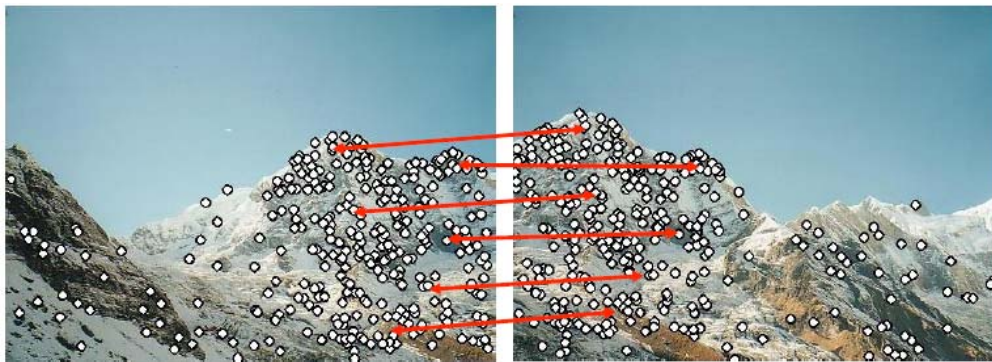
- Efficient
- Rotation

Compare methods



Why extract features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Thanks!